School of Computing and Augmented Intelligence
ASU IRA A. FULTON SCHOOLS OF engineering

CSE446 / CSE598
Software Integration and Engineering

Unit 2
Software Architecture and Integration

Lecture 2-3
Workflow Concepts and Workflow Foundation

Reading: Text Chapter 8, Sections 8.1 and 8.2

Dr. Yinong Chen

Unit 2 Outline

Unit 2-1 Advanced Web App Architecture

Unit 2-2 Enterprise Architecture and Business Process

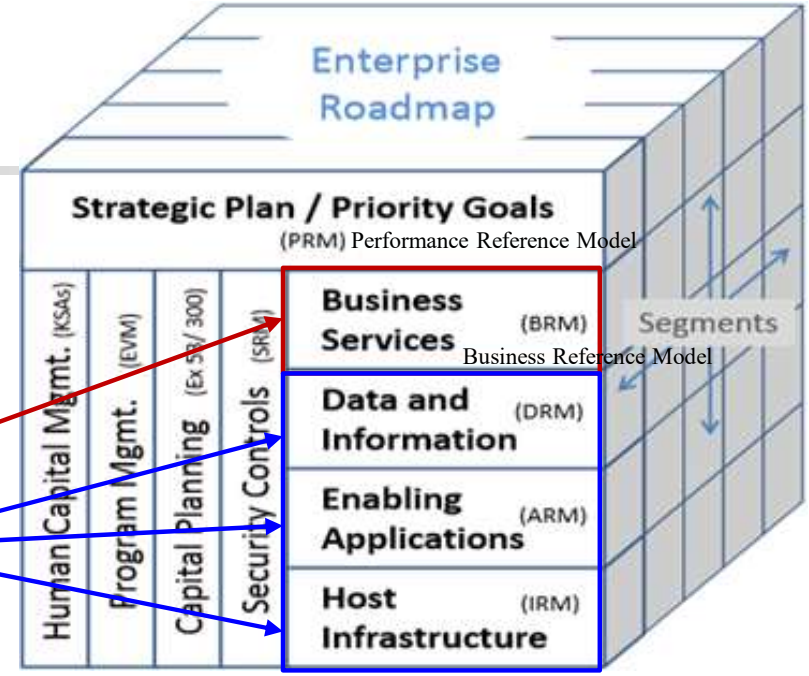
Unit 2-3 Workflow Concepts and Workflow Foundation

Unit 2-4 Software Integration Using Power Platform

Lecture 2-3 Outline

- Previous lecture focused on the architecture design and management
- Workflow for Architecture and Process Integration
  - Cover the Gap between Architecture (CSE360) and Development (Programming Courses)
  - Key Ideas of Workflow-based Application Development
  - Workflow Foundation Constructs and Activities
    - Creating WF Workflow Application
    - Event-Driven Approach and State Machine in WF

What is Workflow?



- Workflow is a new solution to an old problem: Integrating, managing, and supporting business process.
- Workflow offers a new model for the division of labor between people and computer:
  - People do only those that computers cannot: What we want.
- Workflow better matches the business process/logic that customers require, and thus the customers can better understand the solution offered by the software engineers / architects.
- Workflow better separates the tasks of software architects and programmers.
- Graphic (Visual) Programming makes programming easier.
- SOA makes executable workflow possible.

Kepler Project

https://kepler-project.org/

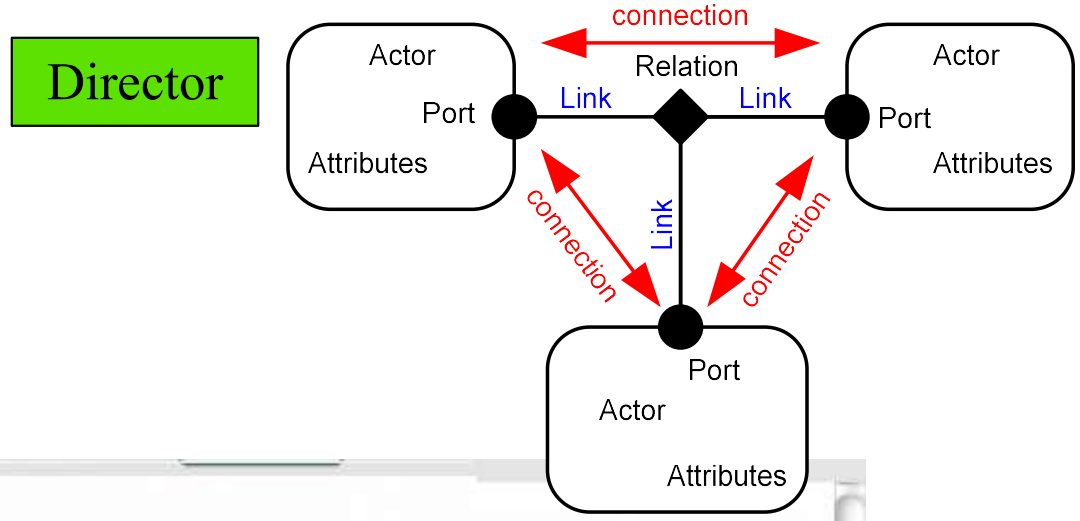
One of the early efforts first released in 2005

- Operate on data stored in a variety of formats, locally and over the internet;
- Integrating disparate software components, such as merging scripts with compiled code;
- Facilitating remote, distributed execution of models.
- Select and then connect relevant analytical components and data sources to create a "scientific workflow" — an executable representation of the steps required for generating results;
- Share and reuse data, workflows, and components developed by the community to address common needs.

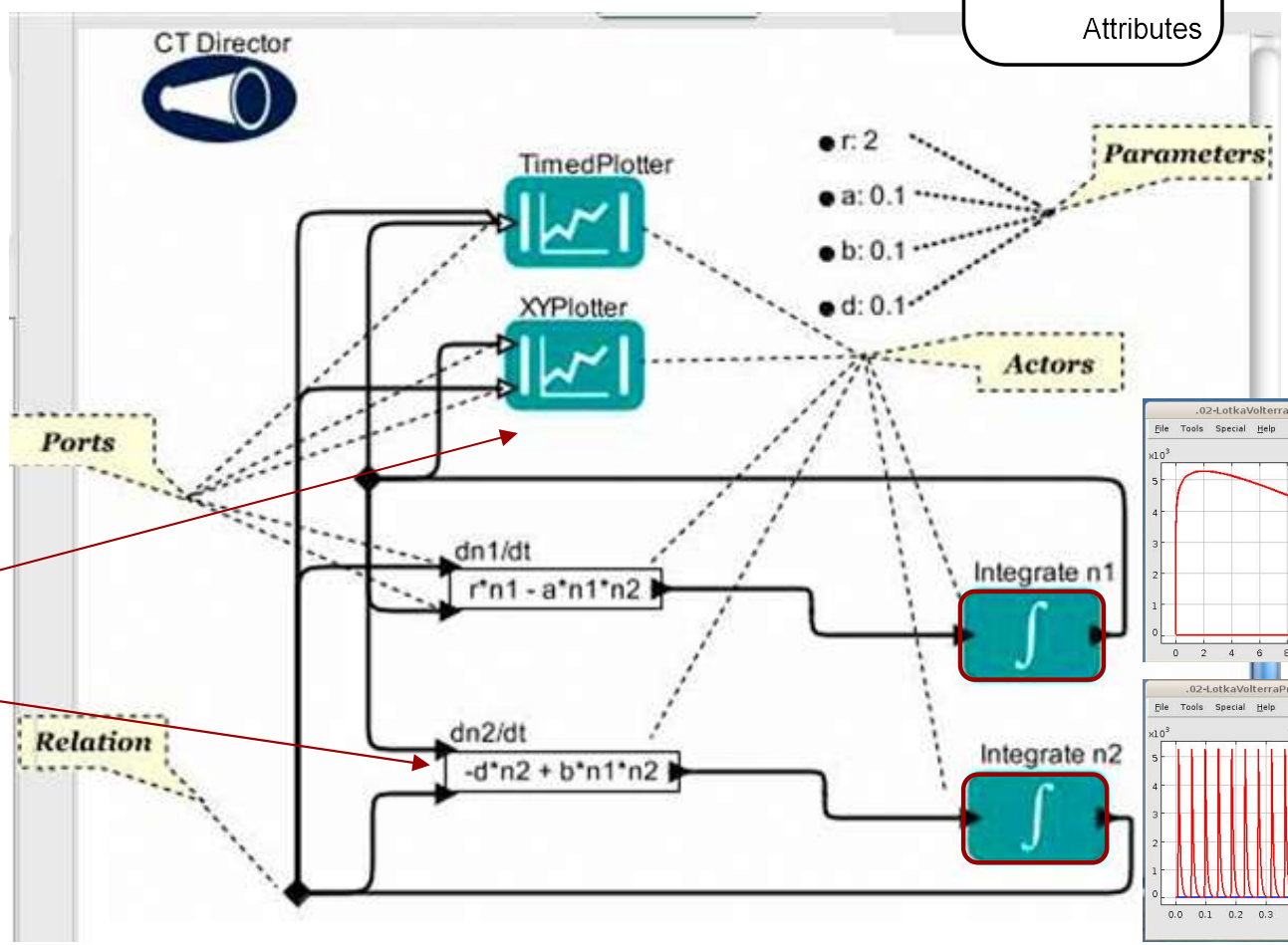
All these ideas are implemented in Power Platform in next lecture.

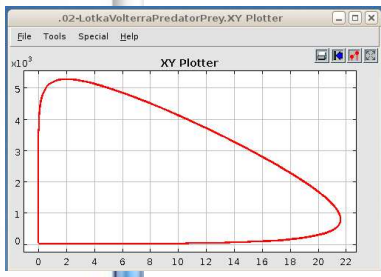
Kepler Workflow System

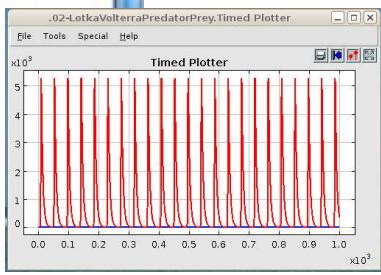
http://edutechwiki.unige.ch/en/Kepler_workflow_system



- Kepler Workflow Style
  - Directors (define computing and communication models)
  - Actors
    - Ports
    - Attributes
  - Relations
- Example: Integration of a template, a data set







Traditional Software Development Process

- Architecture Design
- Flowchart / Process

} Software Architects, Software Engineers

- Coding / Debugging

} Programmers

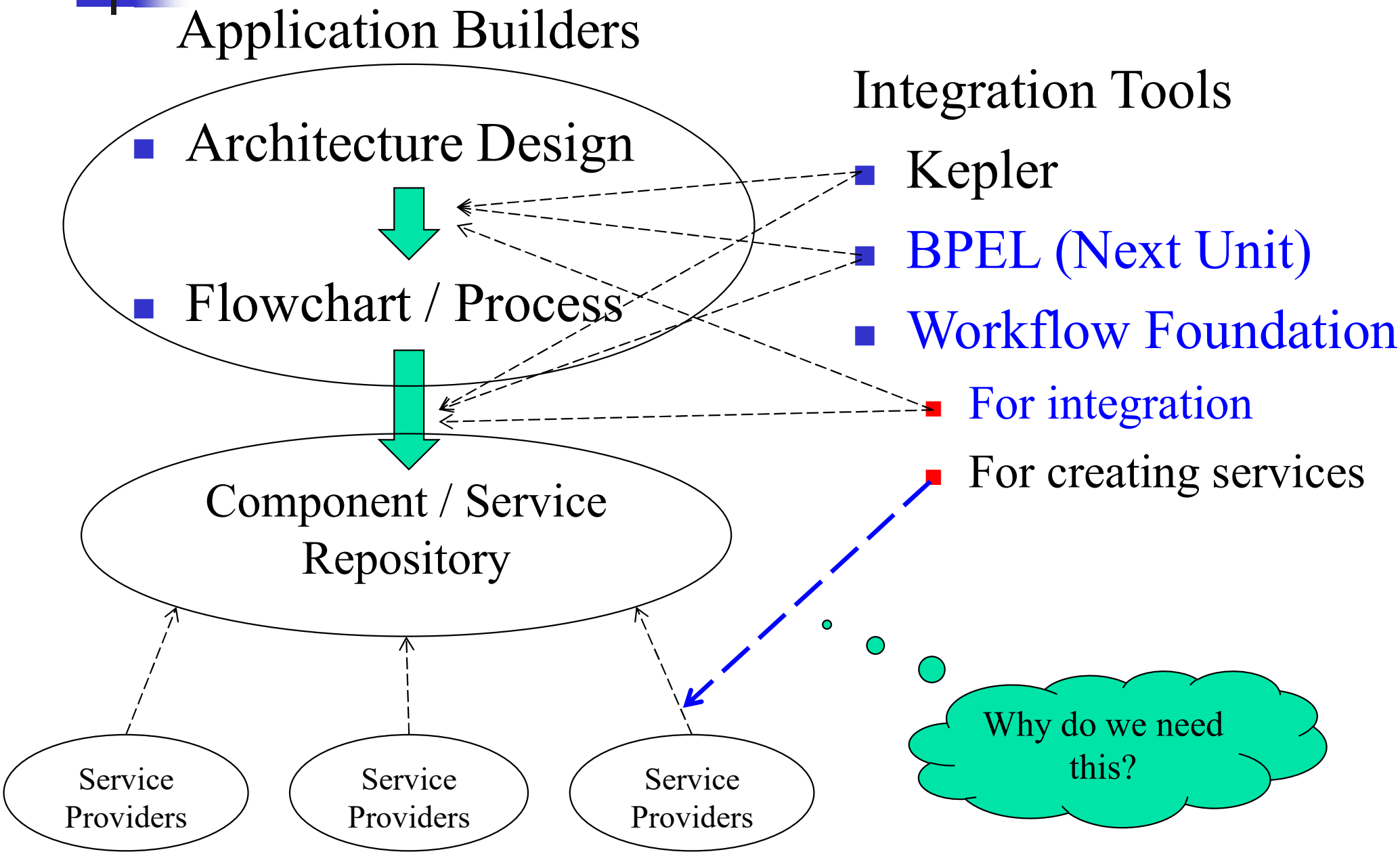
- Testing / Verification

} Testers

- Deployment / Maintenance

} After Sale Supporters

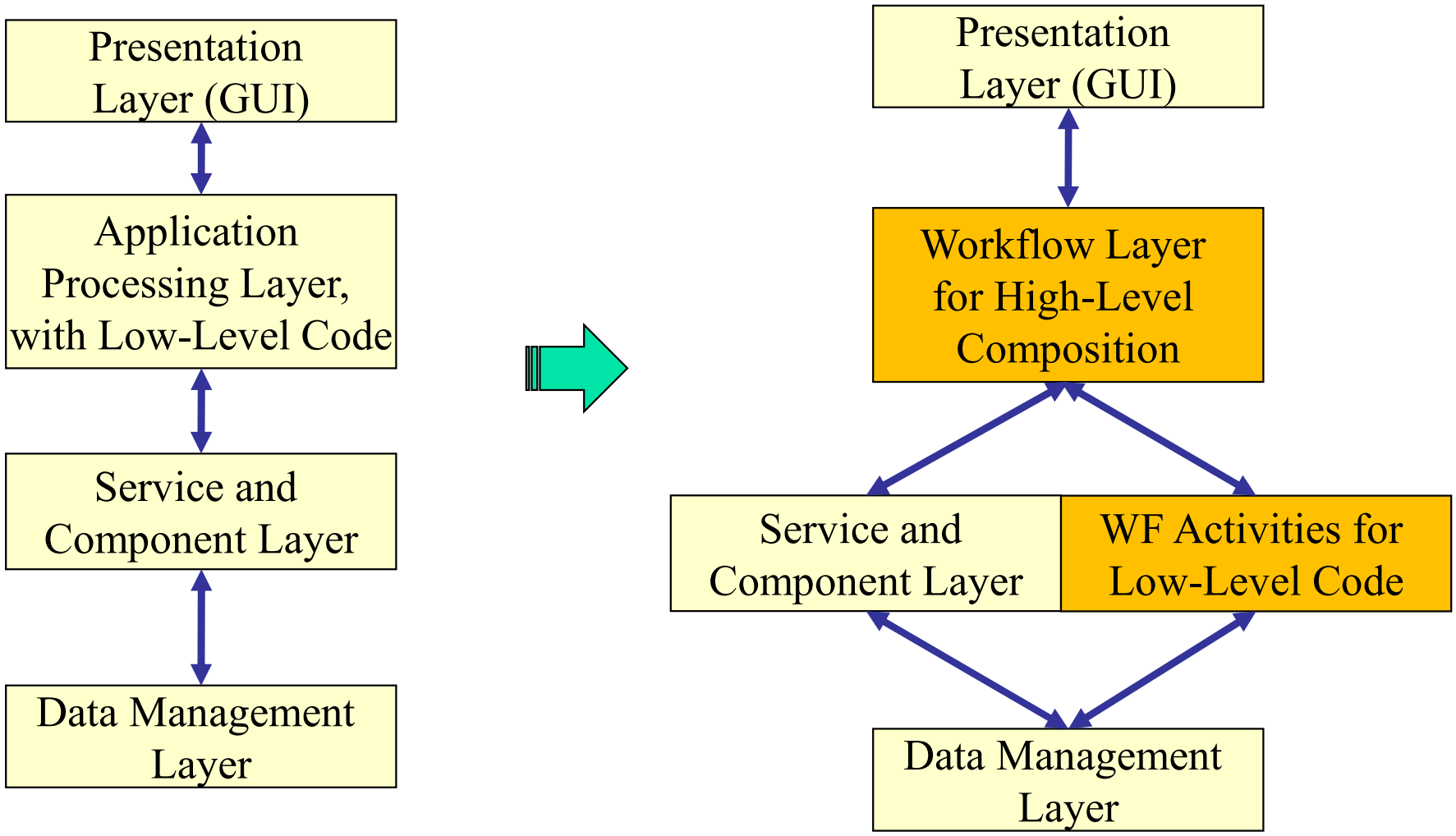
Architecture and Process Integration



Why Do We Need Workflow for Coding?

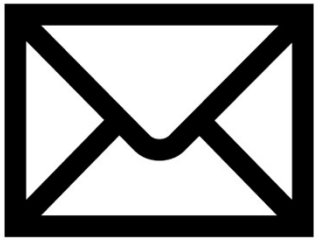
- Low coding/no coding approach: AI makes more feasible.
- Adding a layer of abstraction (graphical approach) to make application development faster and easier.
- It makes the application's main logic more visible.
- By providing a straightforward picture of what is going on, the architect can help developers more quickly to understand an application's structure.
- It helps to maintain the deployed applications, since learning a new application well enough to change it can be a time-consuming process.
- Allows traditional programming style to seamlessly integrate with workflow.

From Tiered Architecture Perspectives



Example: How Do you Write Code to Send a Web Mail to a Client within your Web Application?

```
MService.ServiceClient TestSvcProxy = new MService.ServiceClient();
string EmailAddress = "account@gmail.com";
string Password = "password";
MService.CARRIER PROVIDER = MService.CARRIER.NAME;
string address, Message;
System.Console.Write("Please enter an address: ");
Address = System.Console.ReadLine();
System.Console.Write("Please enter the service provider: ");
System.Console.ReadLine();
System.Console.WriteLine("\nPlease enter a message: ");
Message = System.Console.ReadLine();
bool Result = TestSvcProxy.SMS(EmailAddress, Password,
address, PROVIDER, Message);
if (Result)
    System.Console.WriteLine("\n\nMessage has been sent.");
```



Mail Service

You need to design a GUI page

email address

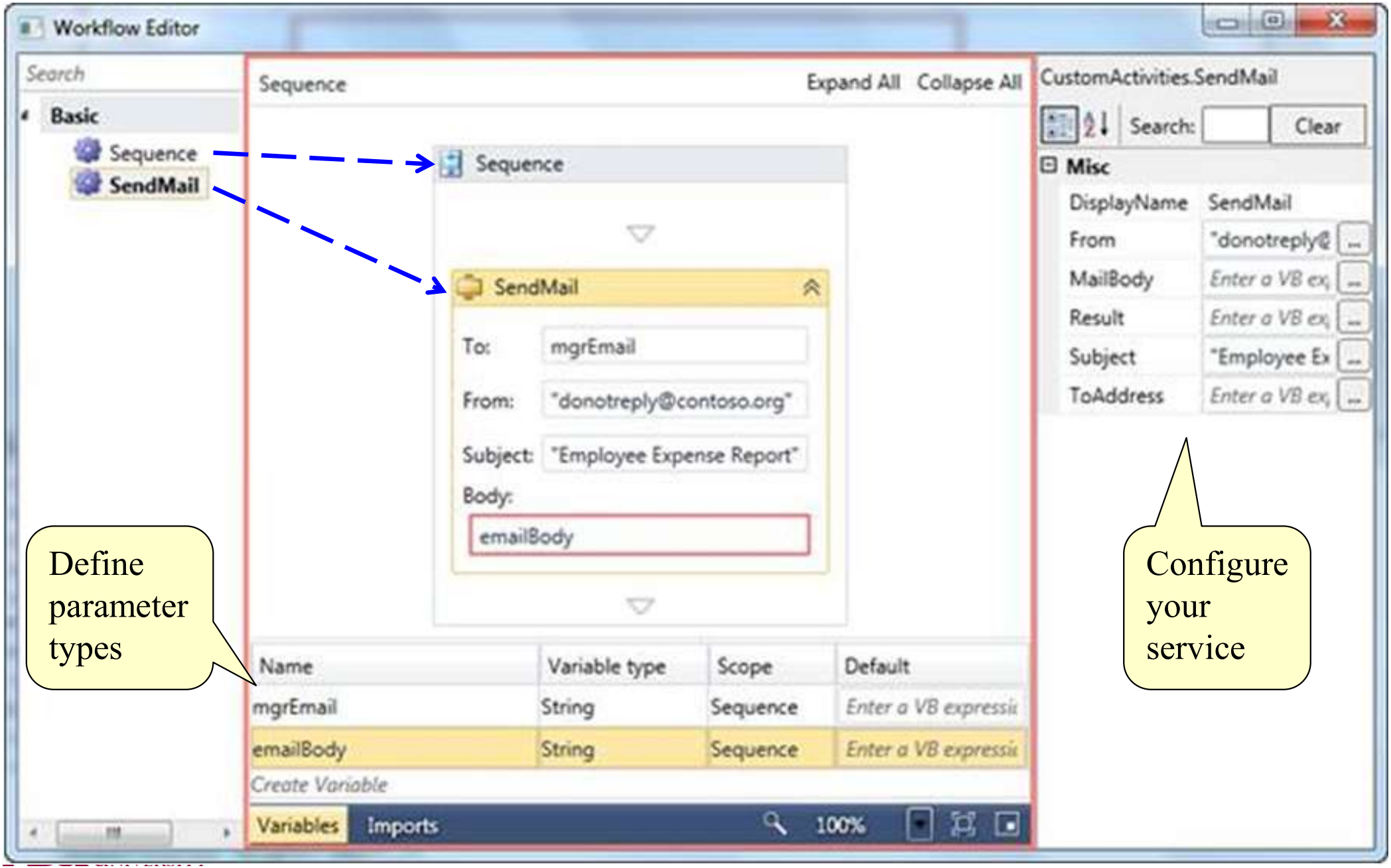
Message

send

Write code behind button

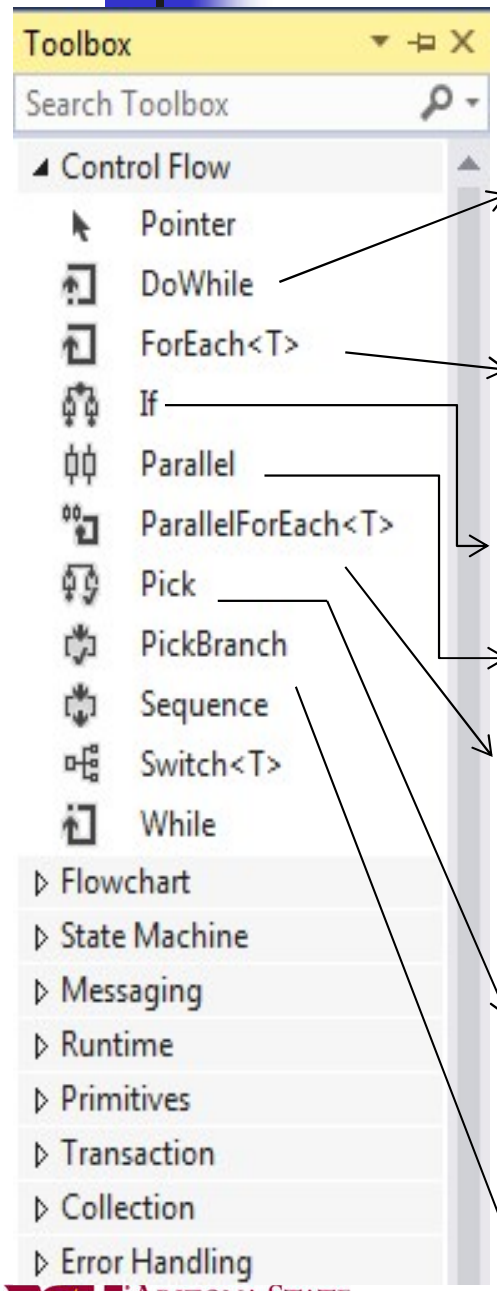
Example: Drag and Drop Designer in WF

http://msdn.microsoft.com/en-us/library/ee342461.aspx



WF Built-in Activities: Control Flow (1)

http://msdn.microsoft.com/en-us/library/dd647759.aspx



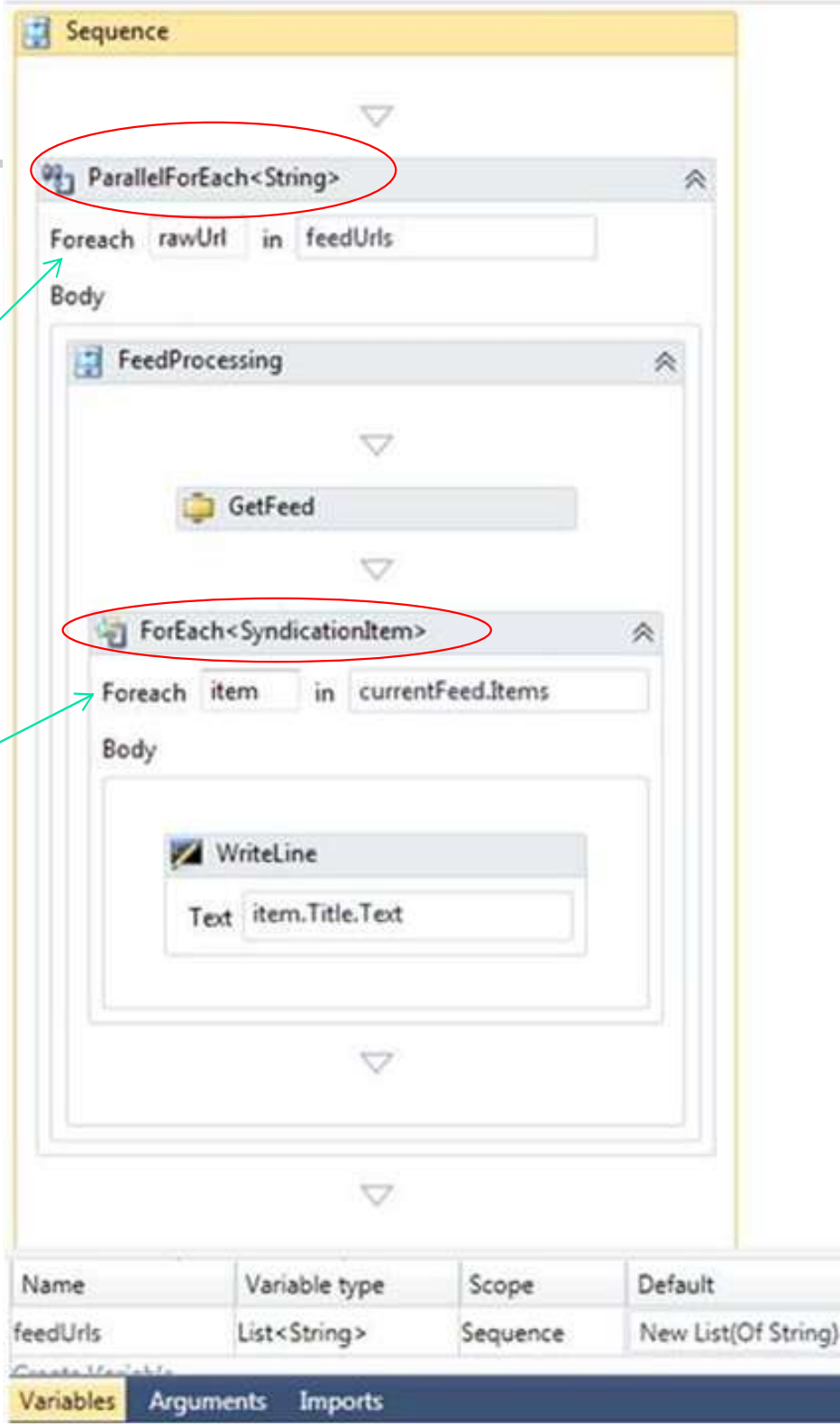
- **DoWhile**: executes an activity, then checks a condition, and repeats until the condition is false.
- **ForEach**: executes an activity for each object in a collection.
- **If**: creates a branch of execution.
- **Parallel**: forks multiple activities.
- **ParallelForEach**: Enumerates the elements of a collection and executes an embedded statement for each element of the collection in parallel.
- **Pick**: allows waiting for a set of PickBranch events, then executing only the activity associated with the first event to occur; event-driven programming!
- **PickBranch**: a member of Pick activity.

ParallelForEach and ForEach

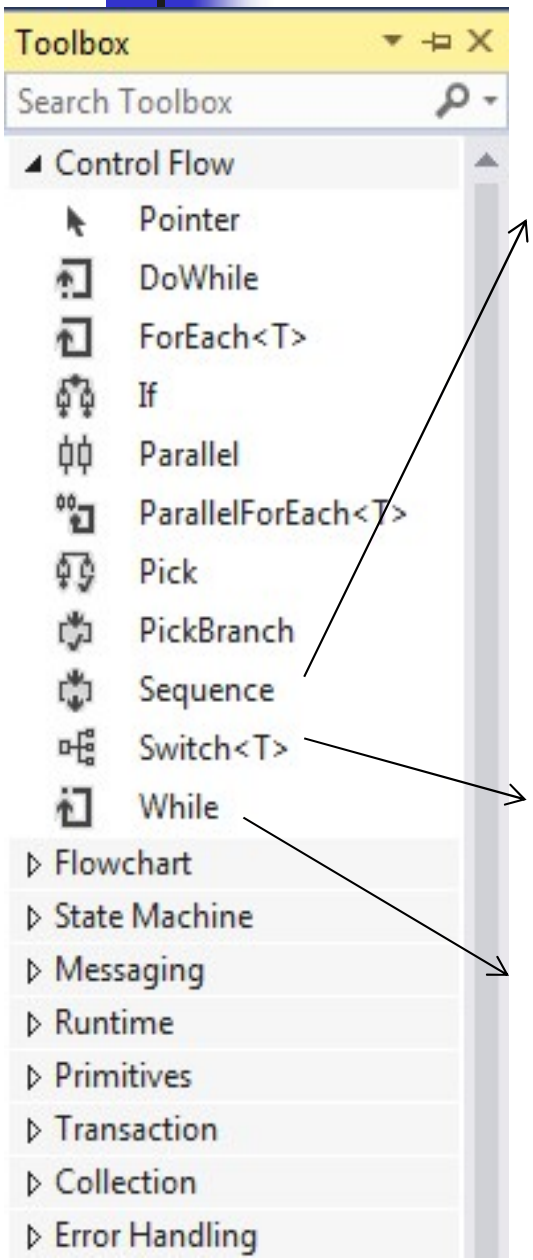
http://msdn.microsoft.com/en-us/library/ee342461.aspx

Example: Call many URLs in parallel:

- Take a list of URLs and asynchronously get all RSS feeds using Foreach <URL>.
- After the feed is returned, the Foreach<item> is used to iterate over the feed items and process them.

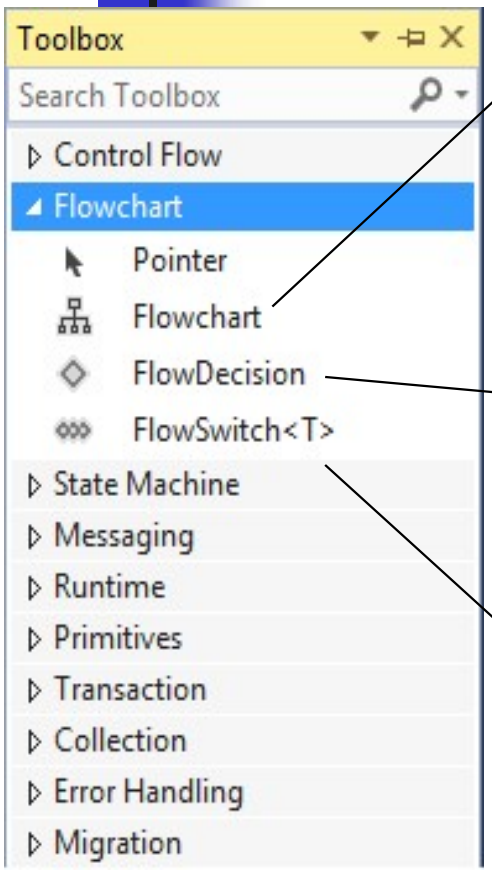


WF Built-in Activities: Control Flow (2)

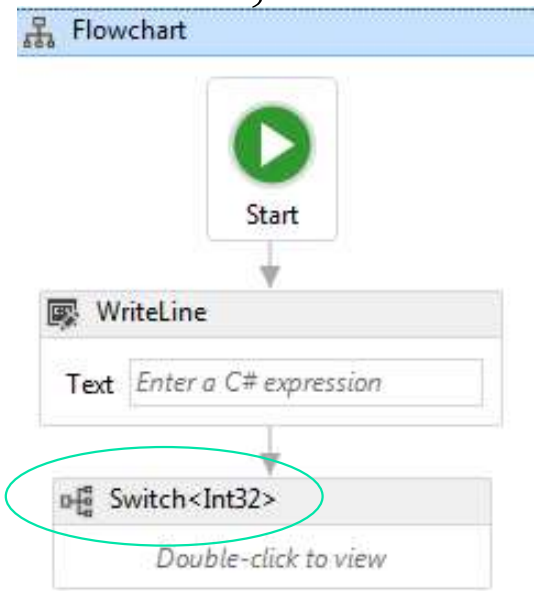


- **Sequence**: groups together a set of activities that are executed sequentially. Sequence is also useful inside workflows. For example, a While activity can contain only one other activity. If that activity is a Sequence, a developer can execute an arbitrary number of activities within the while loop.
- **Switch**: provides a multi-way branch of execution.
- **While**: executes activities iteratively as long as a condition as true.

WF Built-in Activities: Flowchart (3)

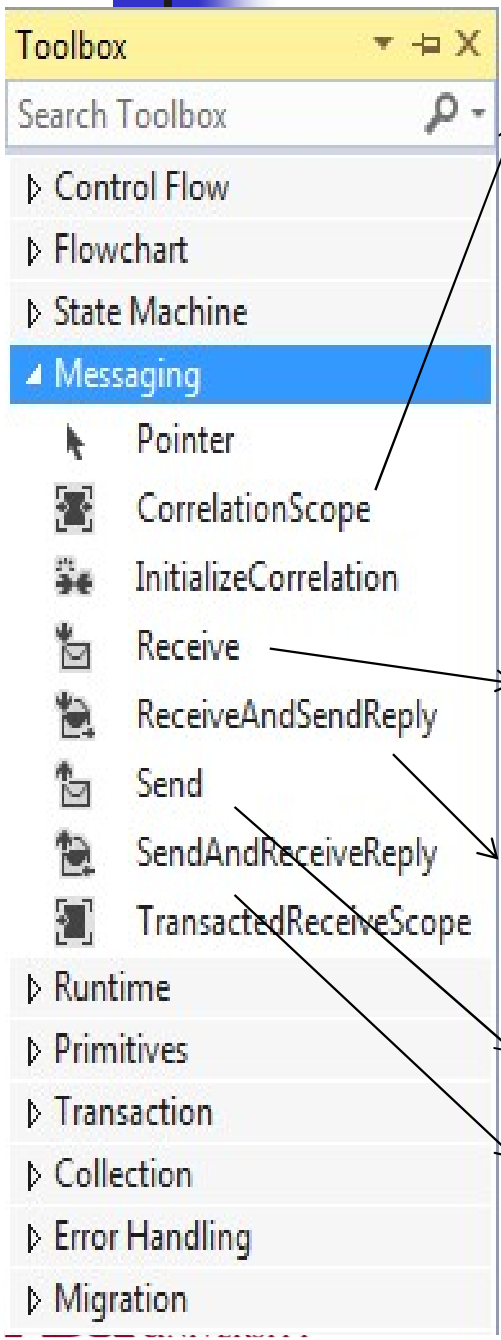


- **Flowchart**: groups together a set of activities that are executed sequentially, but also allows control to return to an earlier step. It creates a local component.
- **FlowDecision**: Branches execution based on a Boolean condition, similar to **If**, but applied at the flowchart level.
- **FlowSwitch**: Branches execution based on an exclusive switch, similar to **Switch** in C#, but applied at the flowchart level.



WF Built-in Activities: Messaging (4)

http://msdn.microsoft.com/en-us/library/dd487617.aspx

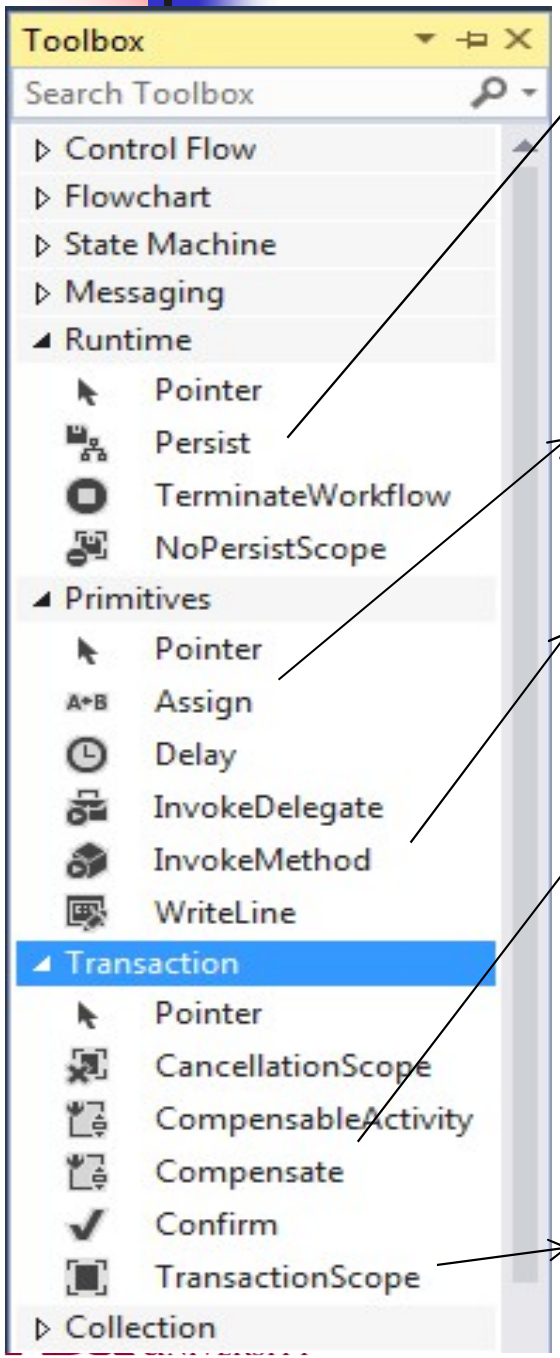


- **CorrelationScope**: Provides implicit CorrelationHandle for child messaging activities. The CorrelationHandle is only visible to child activities, which associates activities together in a correlation by representing a particular shared InstanceKey or transient context in the workflow. Used for asynchronous communication.
- **Receive**: An activity that receives a message via WCF service.
- **ReceiveAndSendReply**: Creates a sequence of two communications.
- **Send**: An activity that sends a message via WCF.
- **SendAndReceiveReply**: Creates an asynchronous communication sequence.

Creating Asynchronous Service in WF

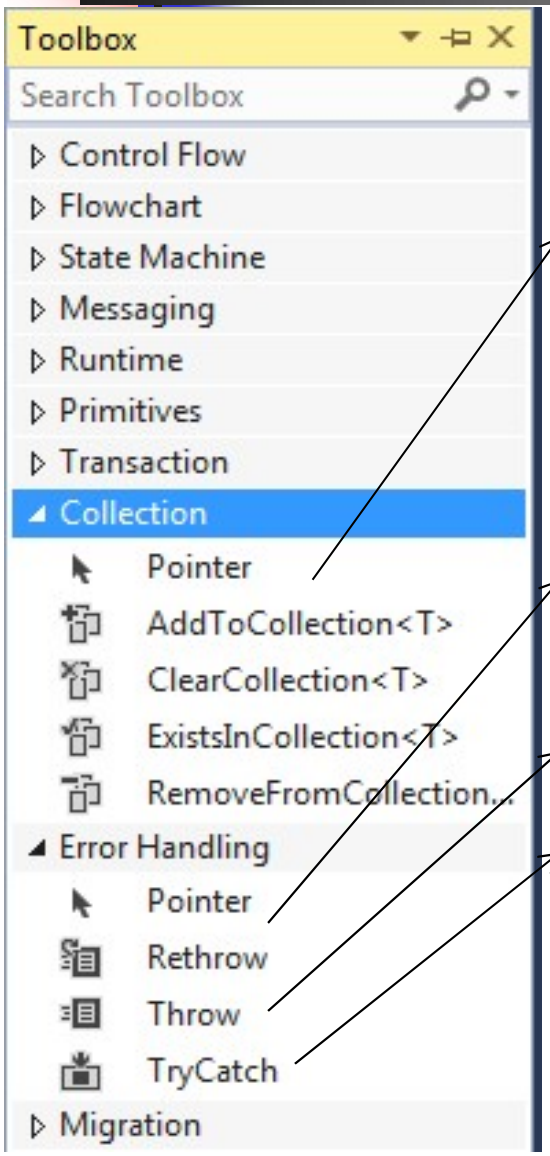
- **SendAndReceiveReply**: is a template used for creating a pair of pre-configured **Send** and **ReceiveReply** activities within a **Sequence** activity
- Send and Receive are correlated as part of an asynchronous request/response message exchange pattern on the client. **SendAndReceiveReply** also
  - Configures the OperationName, ServiceContractName properties of the Send activity.
  - Binds the Request property of the ReceiveReply activity to the Send activity.
  - Creates a CorrelationHandle as a variable in the parent activity.

WF Built-in Activities (5)



- **Persist**: explicitly requests the WF runtime to persist the workflow, to allow a workflow to be re-loaded later on the machine or even on another machine, if necessary. Save all states.
- **Assign**: assigns a value to a variable in the workflow.
- **InvokeMethod**: Invokes a method on the object, synchronously or asynchronously.
- **Compensate**: An activity used to explicitly invoke the compensation handler of a CompensableActivity. It provides a way of handling a problem that occurs in a long-running transaction.
- **TransactionScope:** Makes a code block transactional. This class cannot be inherited.

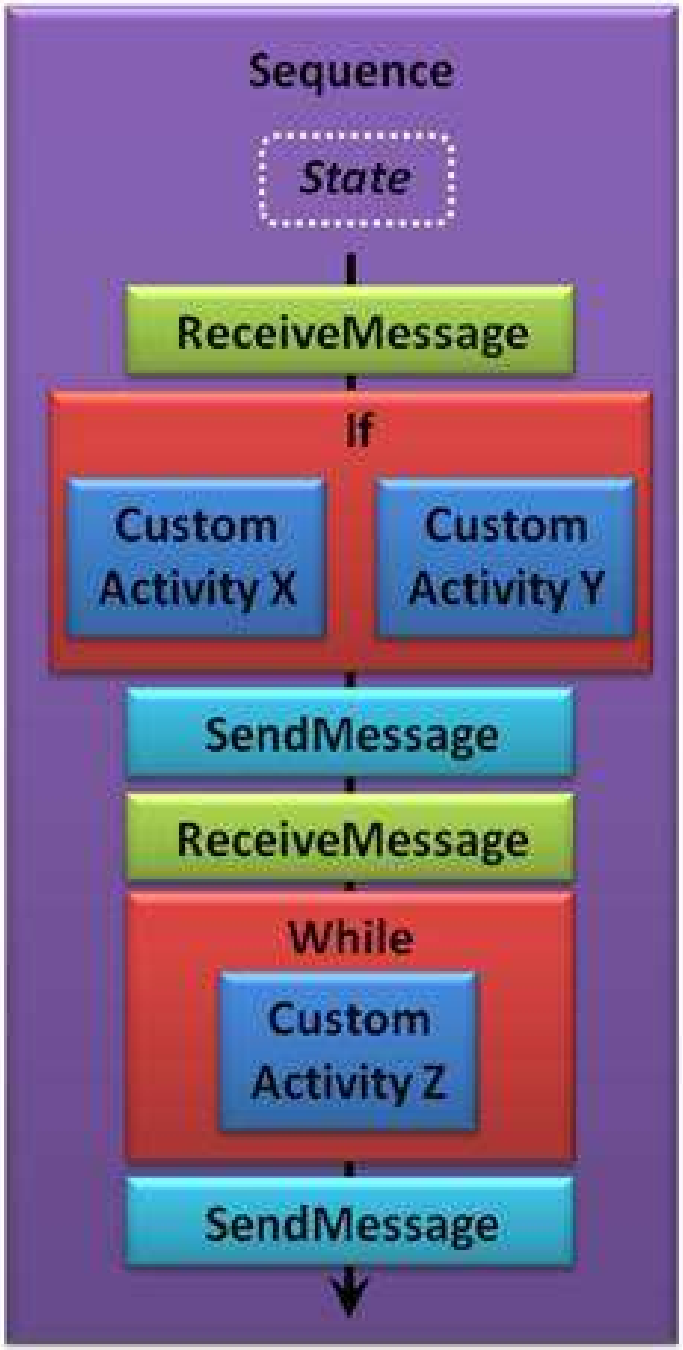
WF Built-in Activities (6)



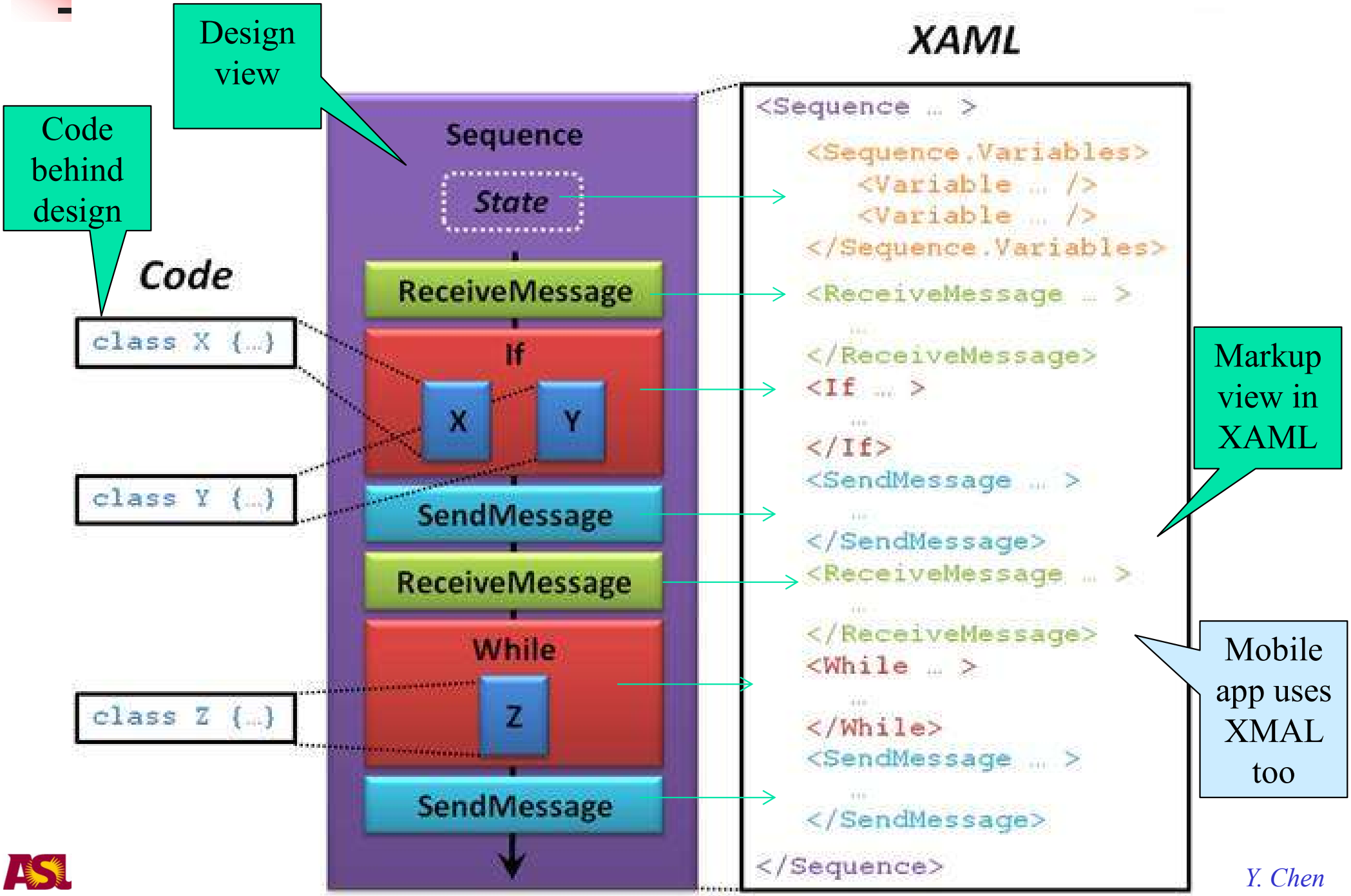
- **AddToCollection**, **ClearCollection**, **ExistInCollection**, **RemoveFromCollection**: allow to add, remove, clear, and check membership of a collection.
- **Rethrow**: throws a previously thrown exception from within a Catch activity.
- **Throw**: raises an exception.
- **TryCatch**: allows creating a try/catch block to handle exceptions. In the block, it contains workflow elements to be executed by the workflow runtime within an exception handling block.

A Typical Workflow of Client Service

http://msdn.microsoft.com/en-us/library/dd851337.aspx



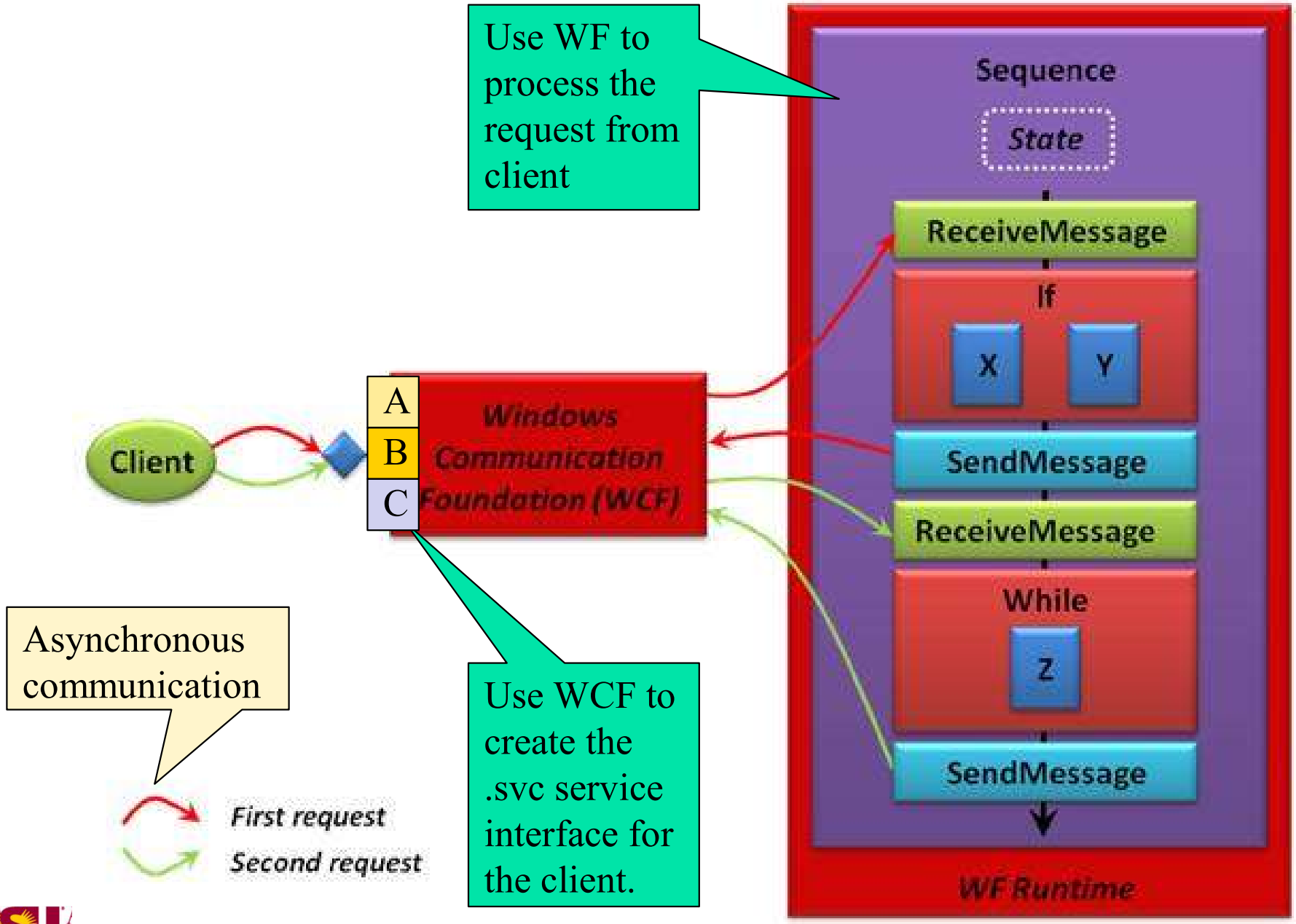
WF Designer, Markup and Code Behind



Creating a Workflow Service

- Building business logic as a service often makes sense, for example, when the same set of operations must be accessed from a
  - Console application
  - Windows forms application;
  - Website application.
- Implementing this logic as a service consisting of a set of operations that can make the service to interoperate with other operations;
- WCF interface can be used for creating SOAP and REST interface for WF service.

WF Using WCF for Creating Services

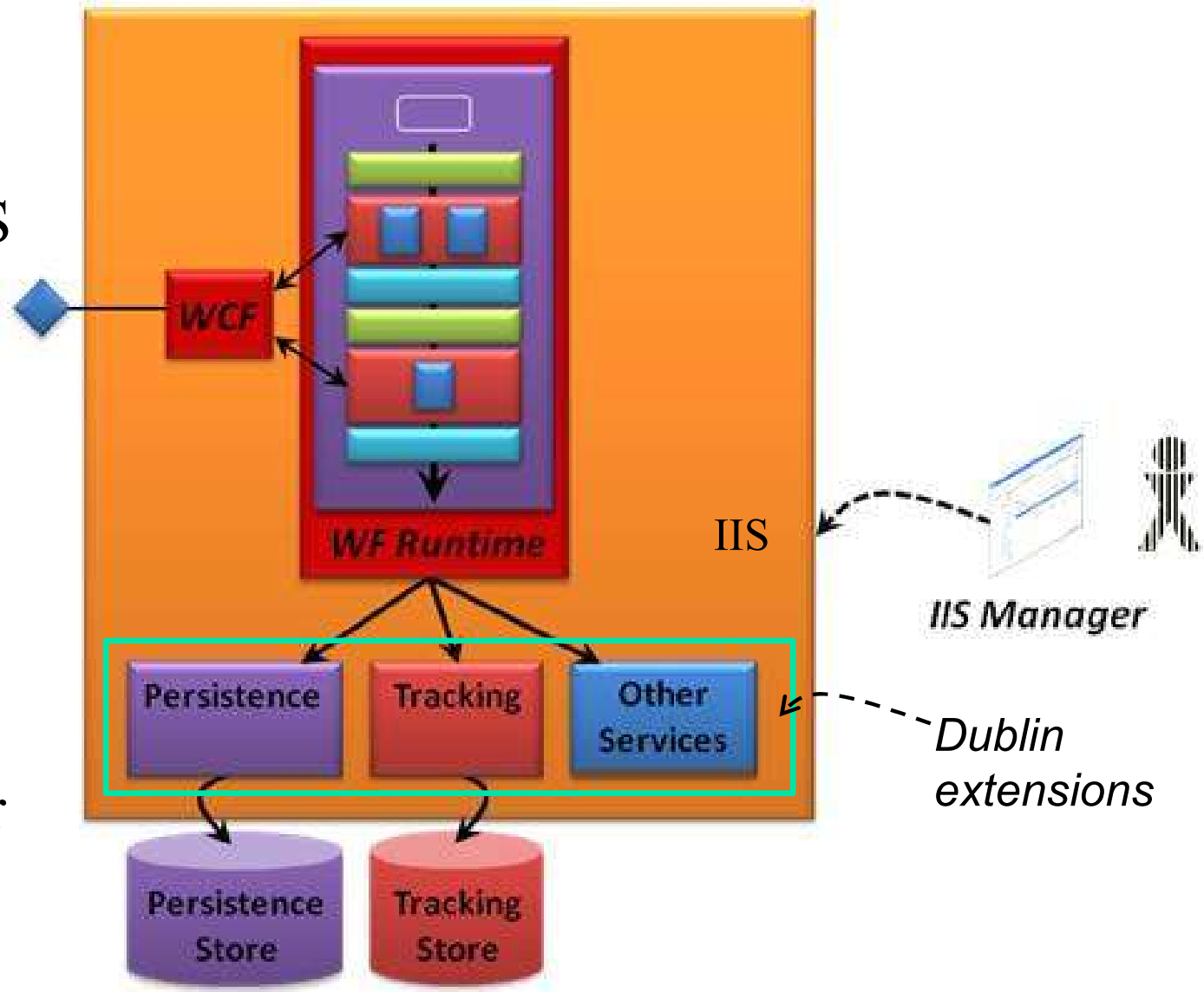


Hosting WF Service

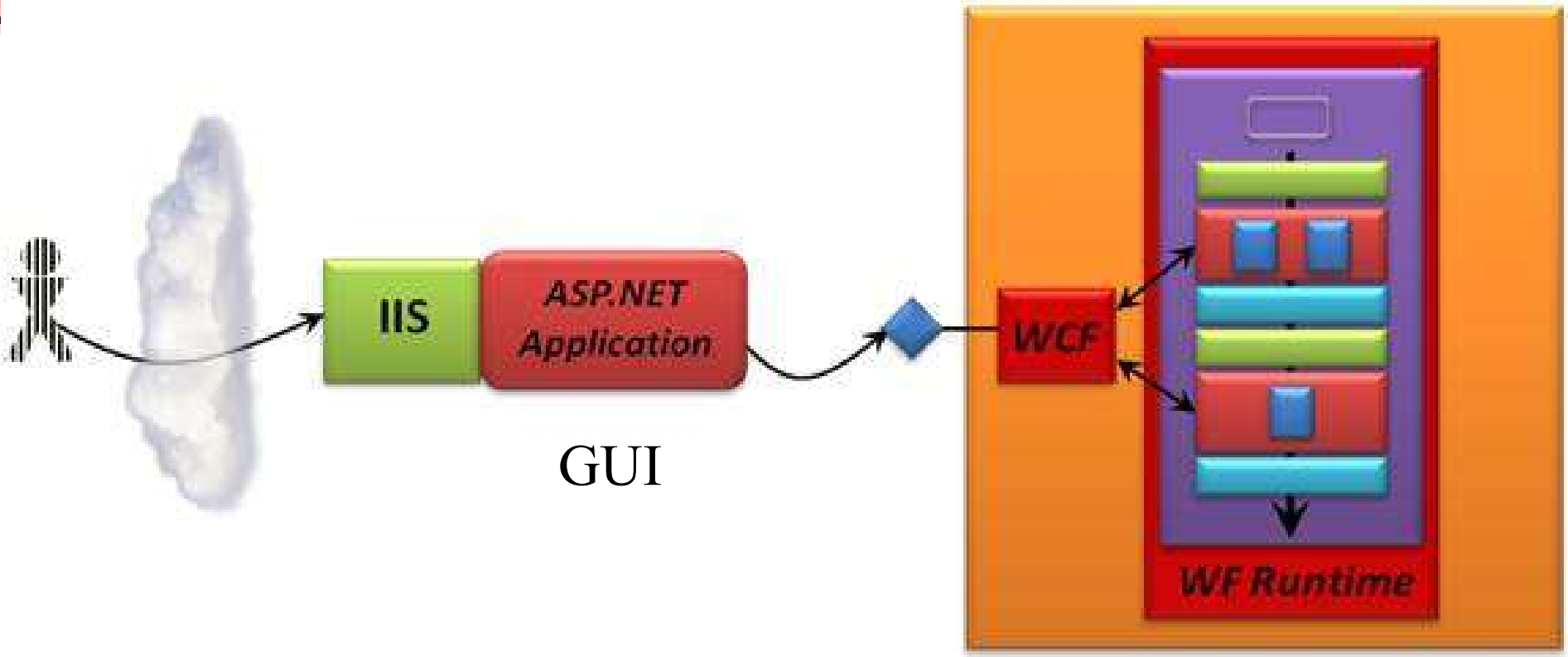
- Similar to WCF, WF services can be hosted in different ways:
  - Self-hosting
  - IIS hosting: running a thread of a worker process
  - "Dublin" hosting
- "Dublin" hosting is based on IIS with extensions for additional support:
  - Persistence management: Saving the process (code and states for later access) and waiting for the second part of service call persistently;
  - Tracking and monitoring
  - Other utility services

Hosting WF Service with Dublin

- IIS provide basic hosting service
- Dublin is the code name for extended IIS and WAS (Website Administration Services) for WF service hosting
- The primary goal of "Dublin" is to make IIS and WAS more attractive as a host for workflow services.
  - Persistence
  - Tracking



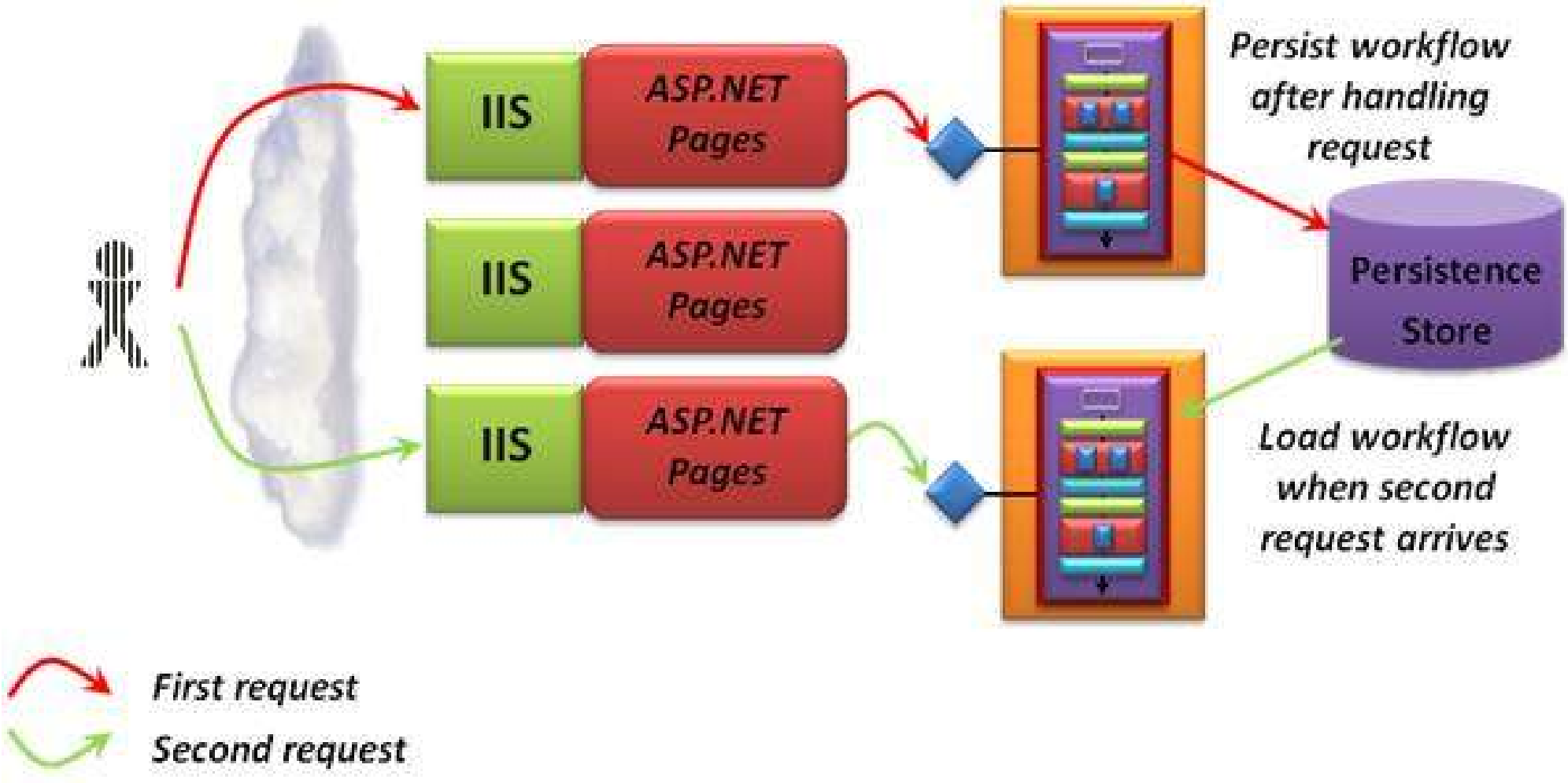
ASP .Net GUI for WF Services



Why do we use WF service behind ASP .Net? We could use code behind ASP .Net page to implement the functionality.

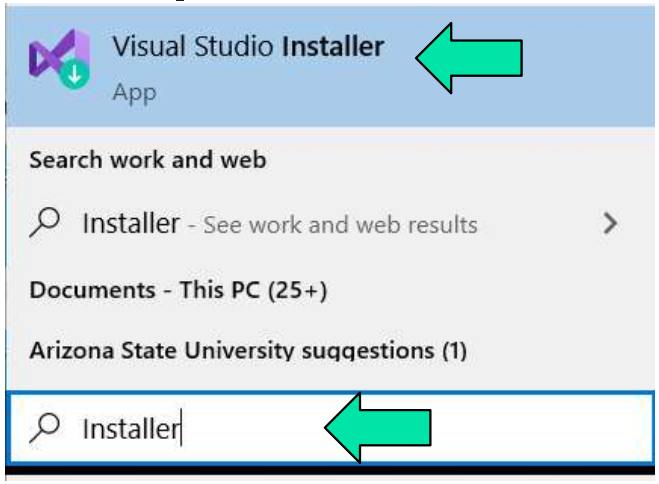
- Tier design: separating presentation layer from application logic layer;
- WF can manage states and asynchronous communication easier than ASP .Net
- With parallel construct and parallel threading, WF is easier for implementing parallel computing.

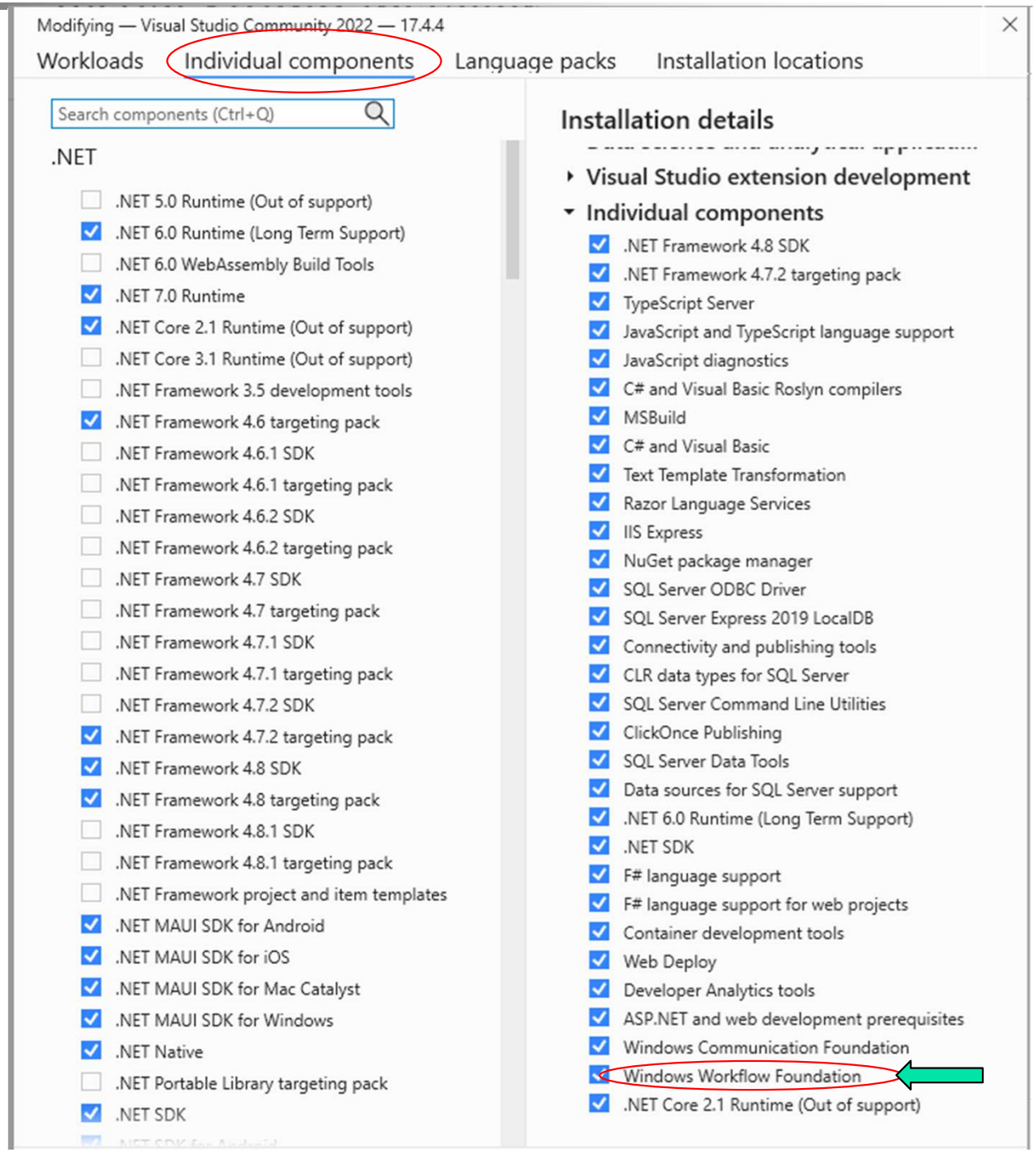
Concurrent Objects for WF Services



Multiple instances can be created for a single client's multiple requests

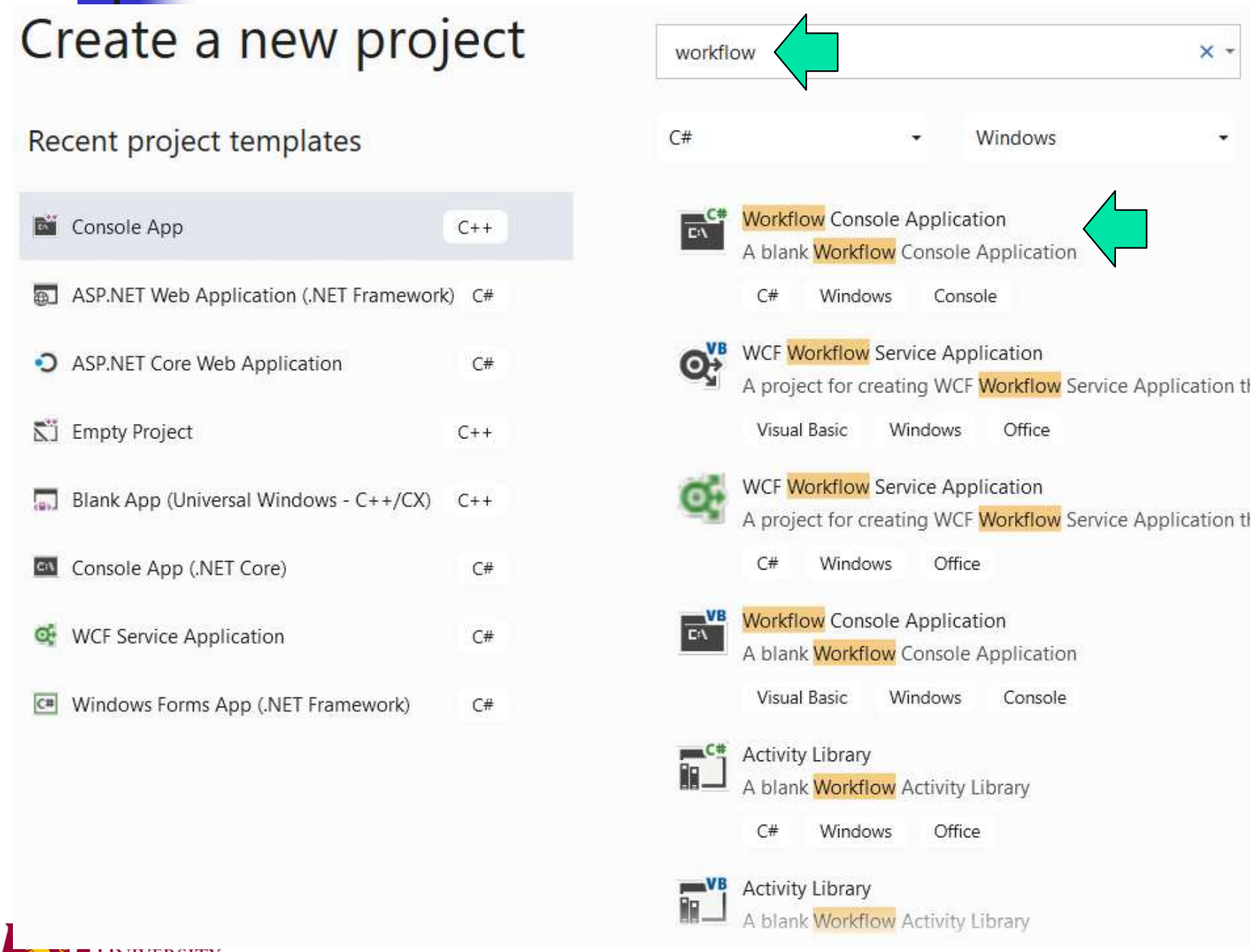
Add Workflow Foundation into Visual Studio 2019/2022





Another way to install: You open the sample WF project in the assignment folder, Visual Studio will detect the missing components and install them automatically.

Getting Started With WF Visual Studio 2022: Hello



Choose Project Name and Location

Configure your new project

Workflow Console Application C# Windows Console

Project name

WorkflowConsoleHello

Location

\Dropbox (ASU)\Current in Dropbox\Teach All\CSE445-446 Programs\Workflow\ContractFirst2025\ ...

Solution name

WorkflowConsoleHello

☑ Place solution and project in the same directory

Framework

.NET Framework 4.7.2

Back Create

Project Created: Main Class

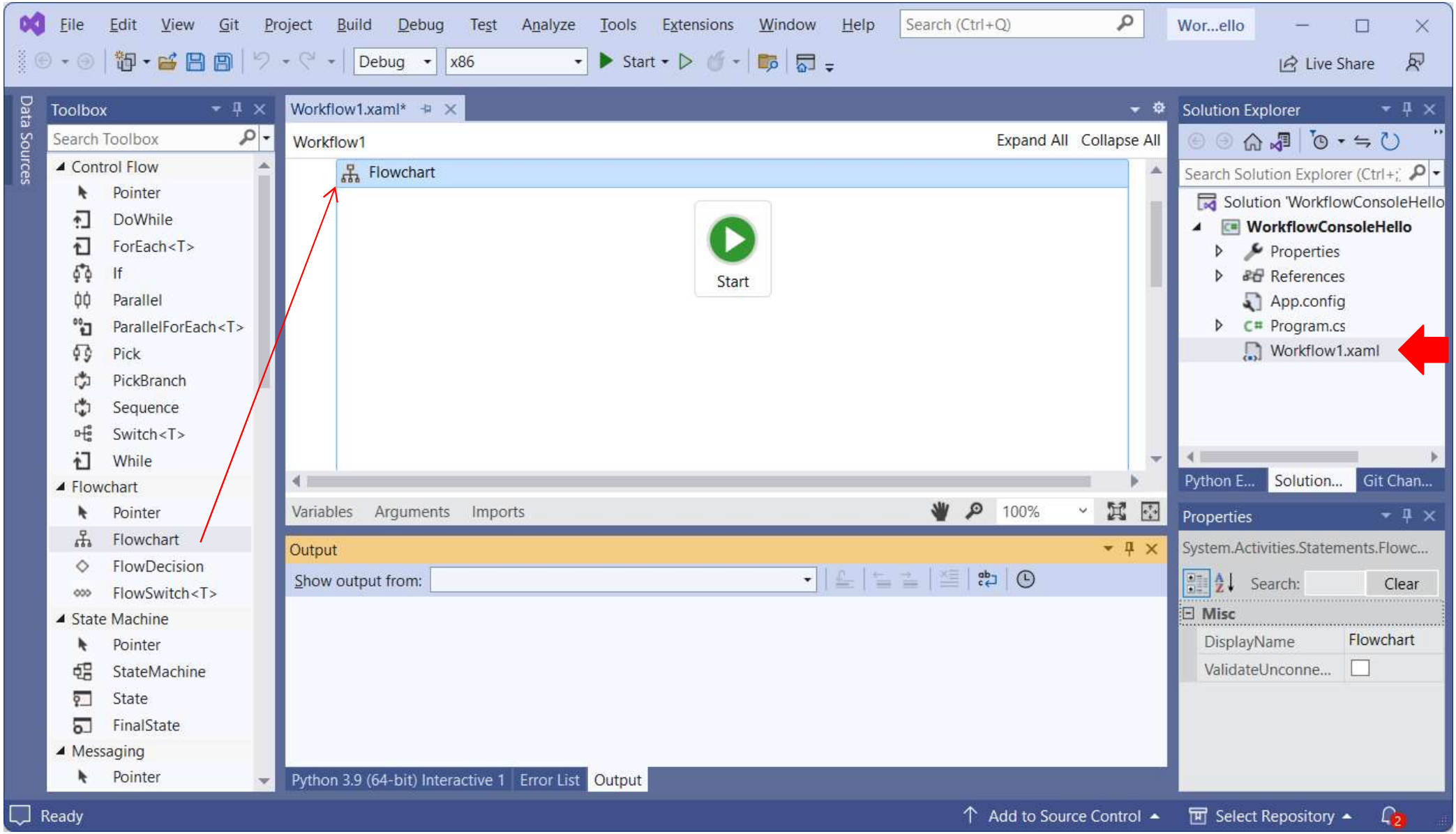
```
using System;
using System.Activities;
using System.Activities.Statements;
using System.Linq;

namespace WorkflowConsoleHello
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Activity workflow1 = new Workflow1();
            WorkflowInvoker.Invoke(workflow1);
        }
    }
}
```

The workflow is considered a thread and started by the Main method.

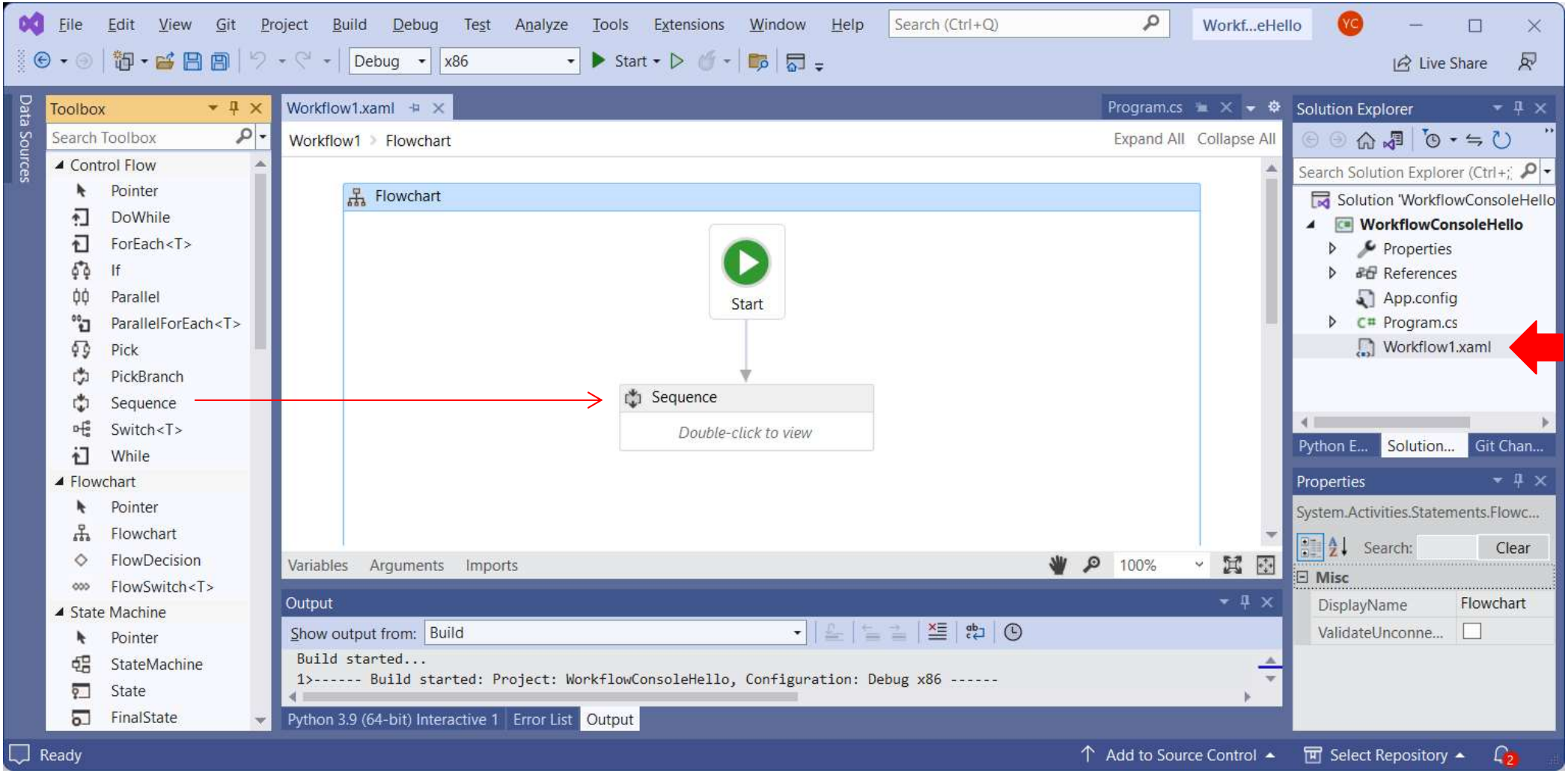
Adding a Flowchart Item into Workflow

1. Drag and drop Flowchart into the design surface
2. Right click project name “WorkflowHelloWorld” and Choose “Add New Item”



Adding a Sequence into Flowchart

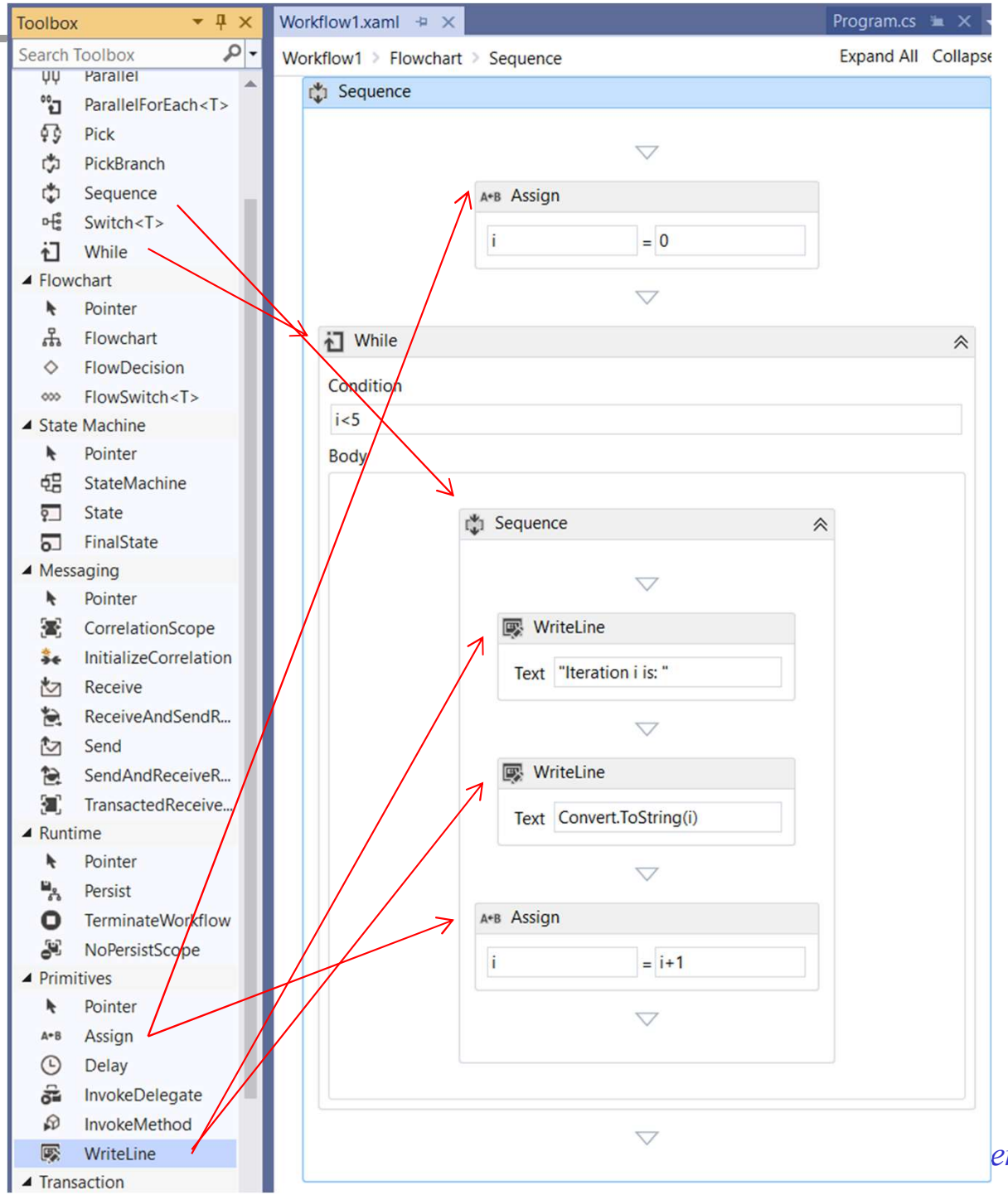
2. Drag and drop Sequence into Flowchart. Make sure that an arrow appear from Start to Sequence
3. Double-click Sequence to view (open) Sequence



Adding activities into Sequence

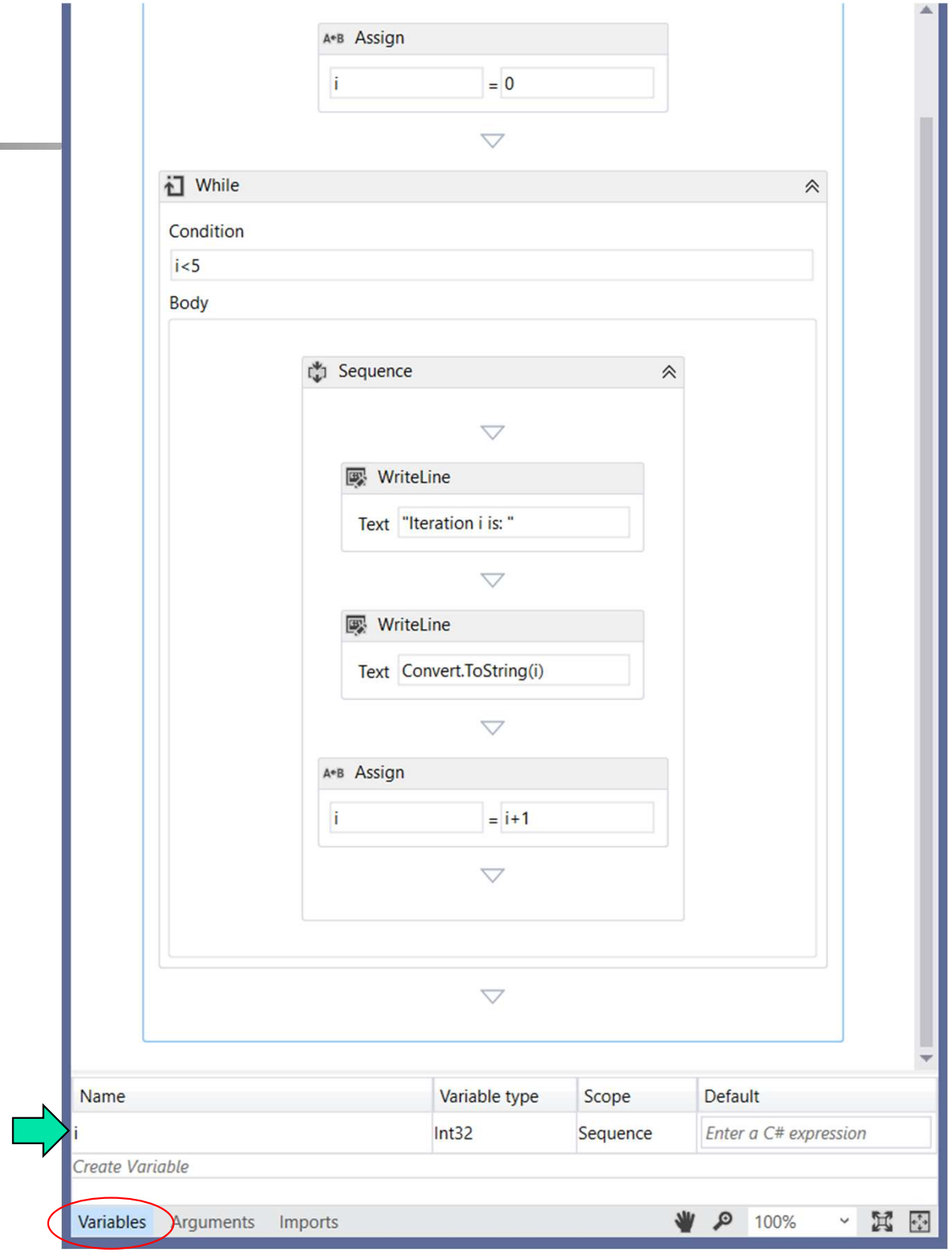
4. After expanding Sequence add the activities into the sequence.

Note the nested structure and the layers of the activities.



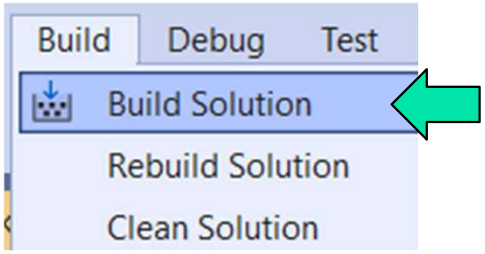
Define Variable

5. We use an integer variable i in the while loop. We need to define the variable.
6. Click Variables" at the bottom of the design surface.
7. Create variable I and choose type to be Int32

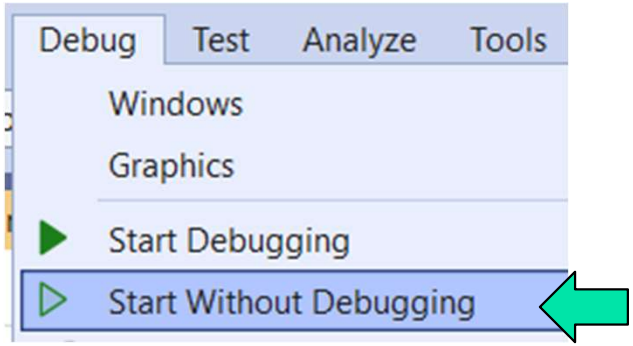


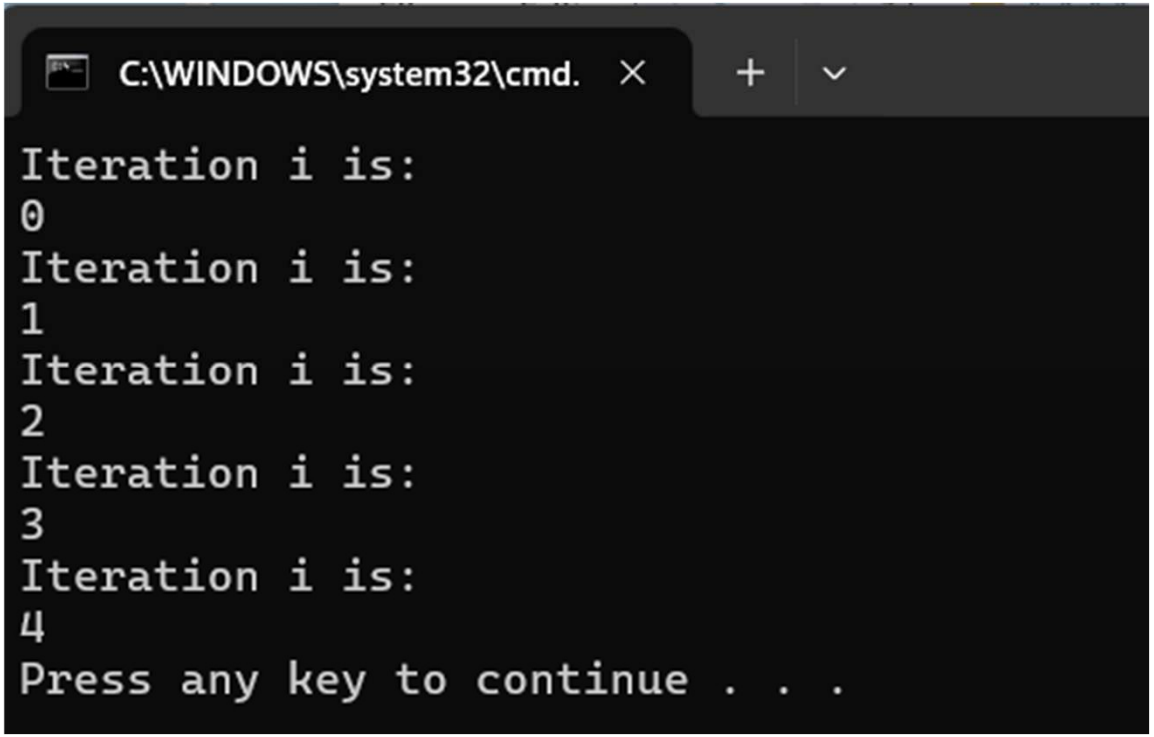
Build and Start without Bebugging

Build the project



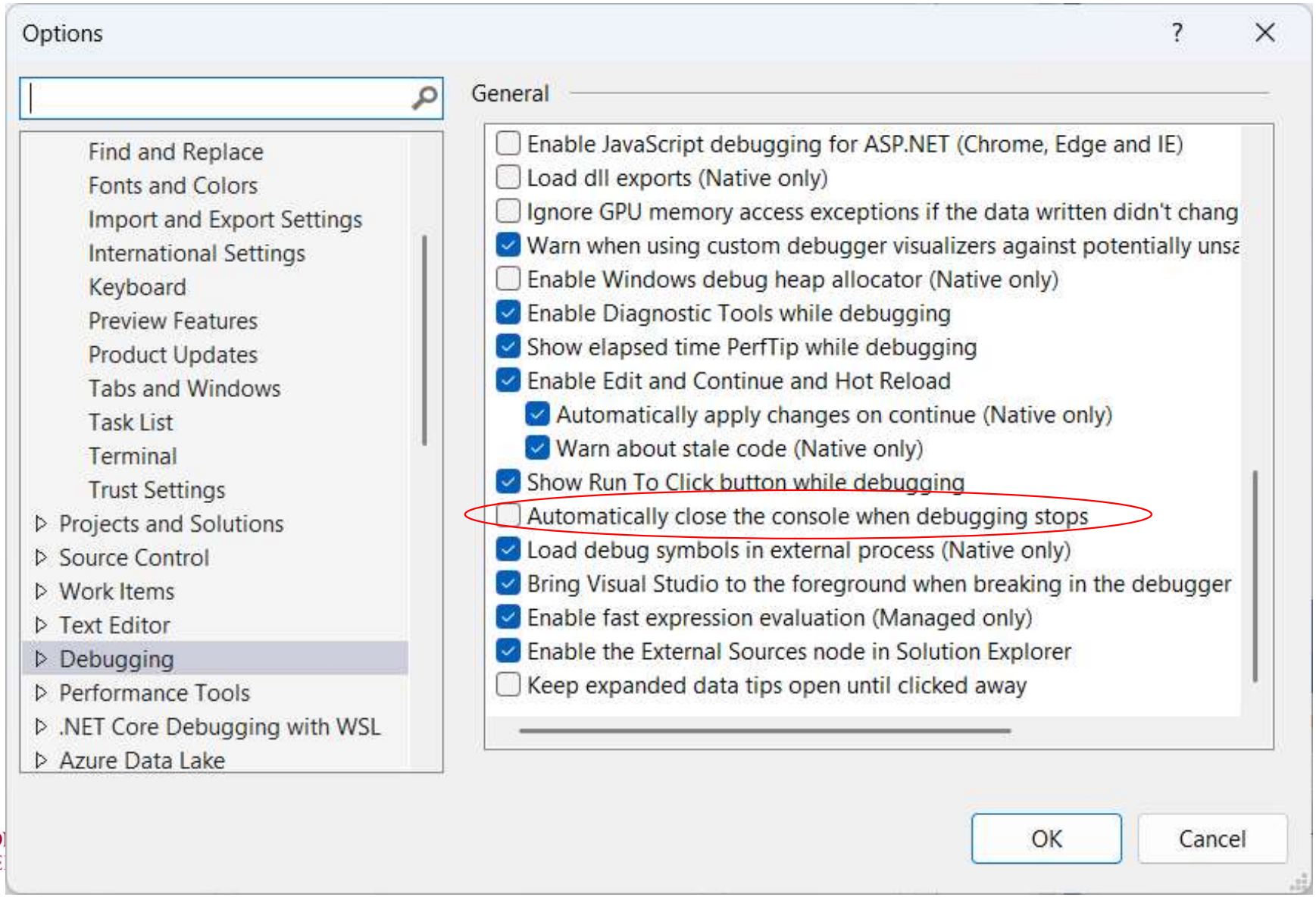
Start without Debugging





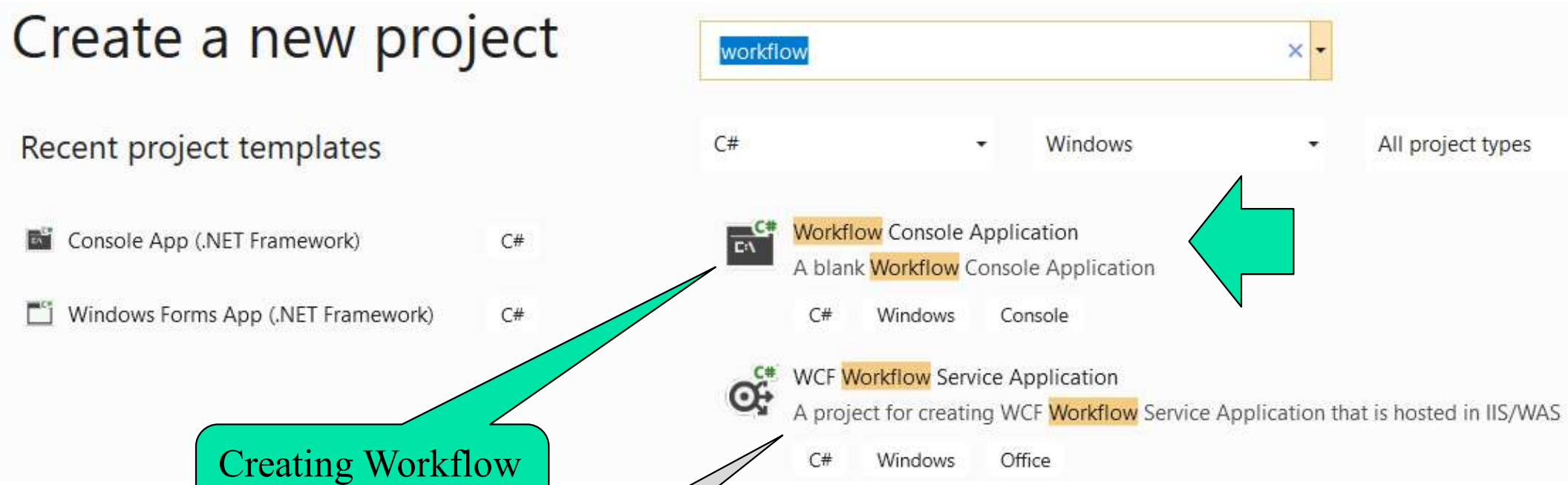
Keep Console Window Open

If the Console window closes automatically after execution, do the followings:
Visual Studio Menu:
Tools → Options → Debugging



A More complex Example

In Visual Studio 2019 (FSE Apporto Virtual Lab)



Creating Workflow Application, now!

Creating Workflow Web Service, next.

Workflow Console Application

In Workflow Console Application template,
The Main() method in Program.cs is the entry point
The workflowInvoker invokes the Workflow1.xaml file

Program.cs | Workflow1.xaml

WorkflowConsoleAppSer | WorkflowConsoleAppSer | Main(string[] args)

```
using System;
using System.Linq;
using System.Activities;
using System.Activities.Statements;

namespace WorkflowConsoleAppSemester
{

    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Activity workflow1 = new Workflow1();
            WorkflowInvoker.Invoke(workflow1);
        }
    }
}
```

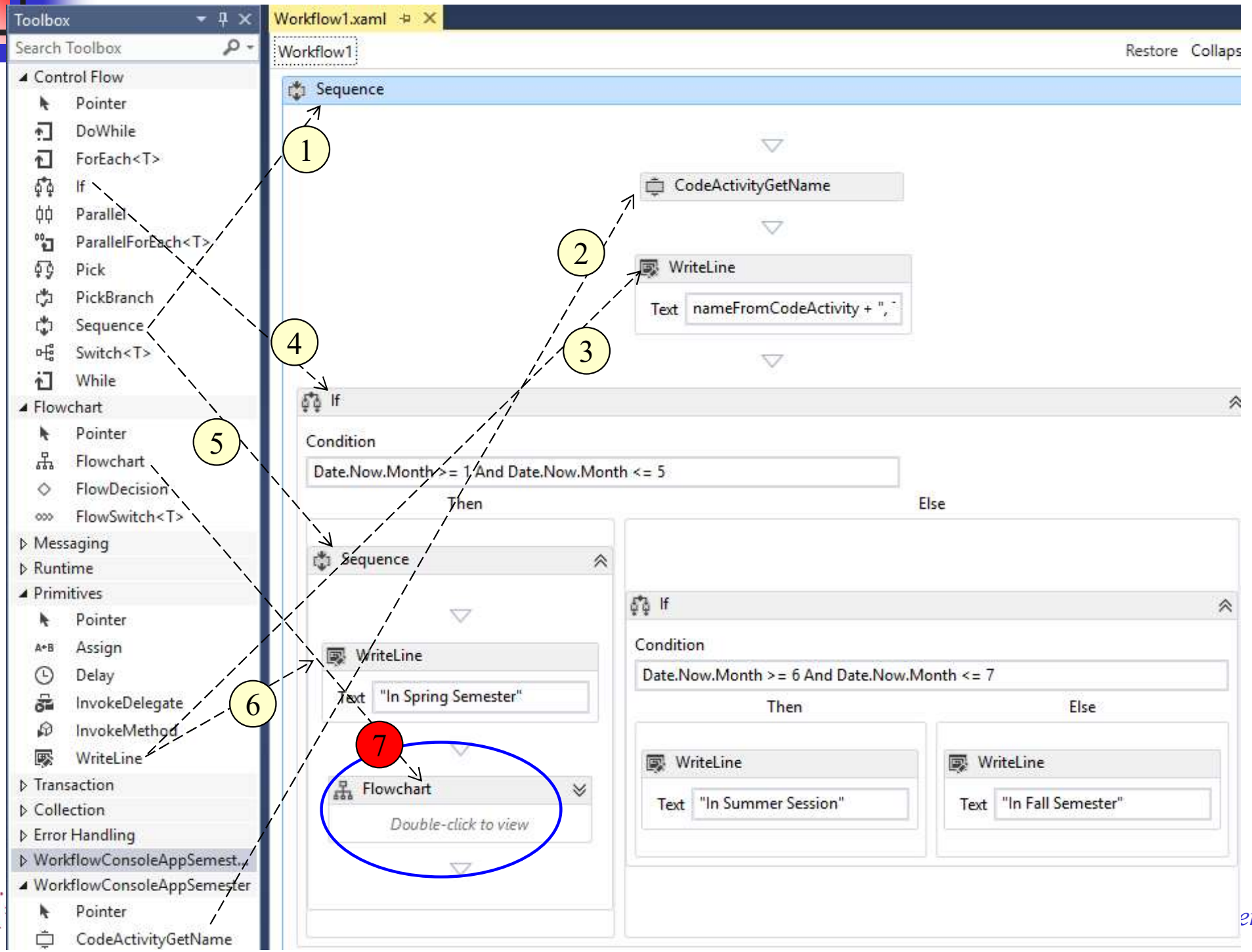
Solution Explorer

Search Solution Explorer (Ctrl+;)

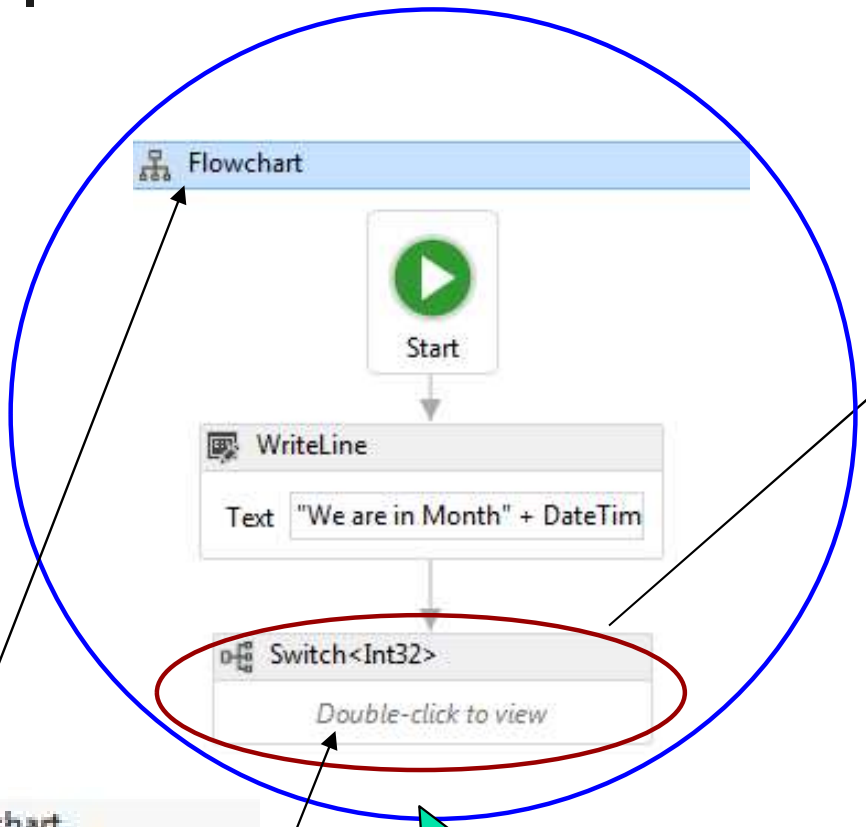
- Solution 'WorkflowConsoleAppSe
- WorkflowConsoleAppSemes
  - Properties
  - References
  - App.config
  - Program.cs
  - Workflow1.xaml

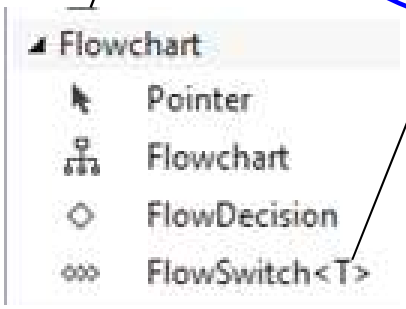
The workflow is considered a thread and started by the Main method.

Graphic Development of Workflow.xaml

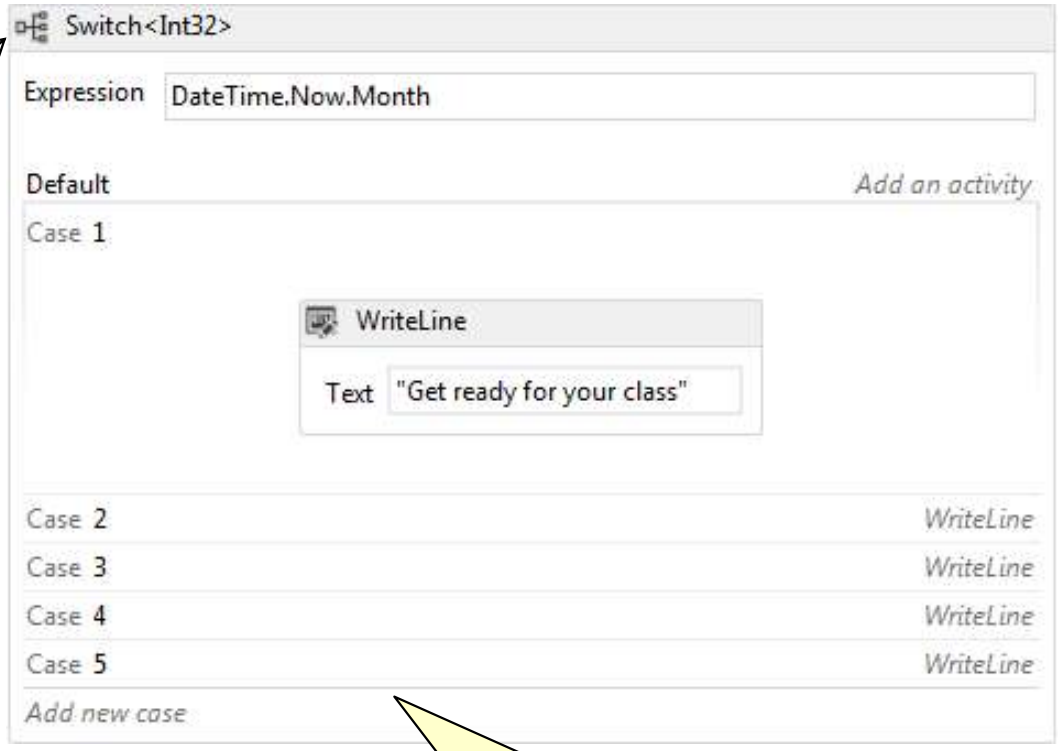


A Flowchart is an Instant Component





A **Flowchart** creates an instant component in a workflow! The custom component does not appear in toolbox

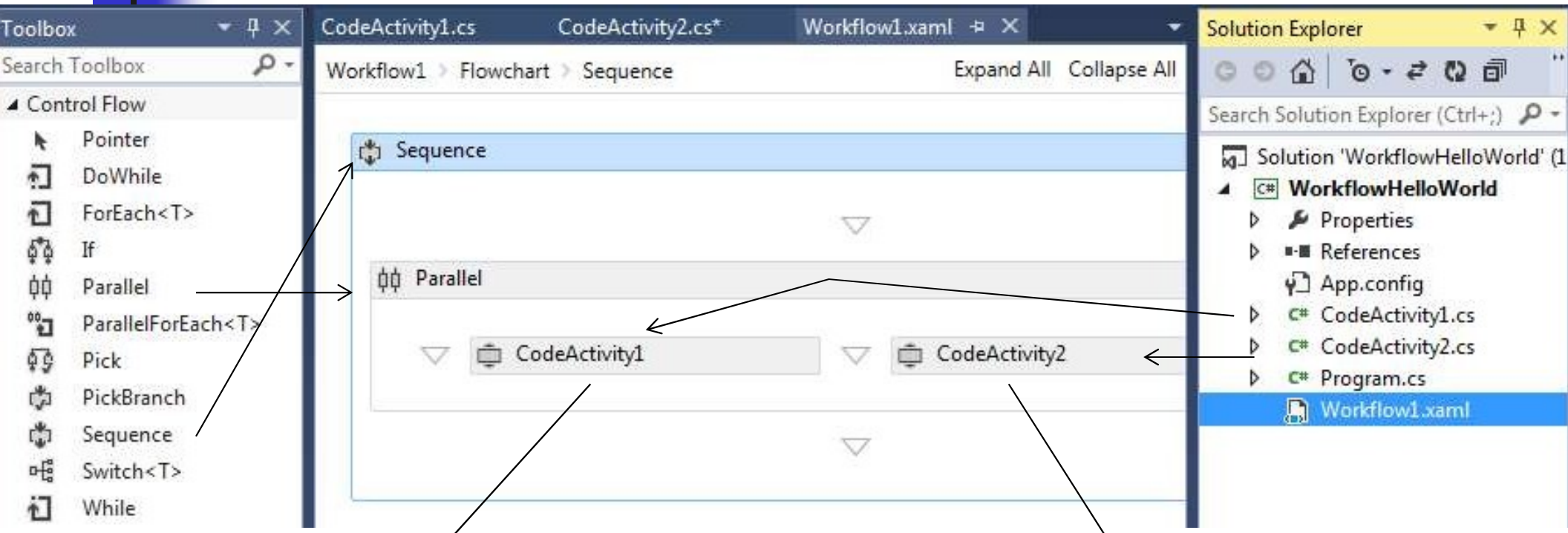


A **Switch** is a construct only. It has a fixed structure.

Workflow: Support Control Flow and Data Flow

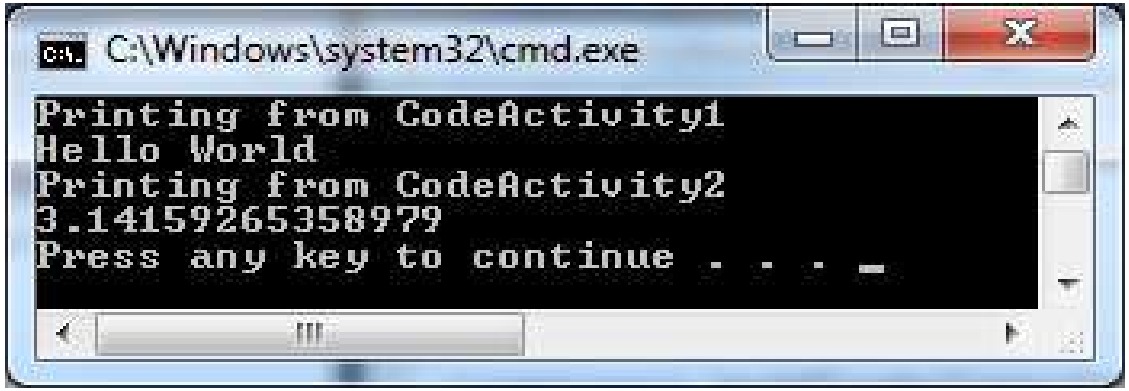
- Control flow model: Assuming one processor will process the entire program, and thus, the code are organized in a **sequence**;
  - It is possible to use multithread programming to perform parallel computing, but not easy.
- Data flow model: Assuming multiple processors will process different part of the program. A piece of code will be executed when the input **data** is ready.
- Workflow supports multiple computing models
  - Sequence construct
  - Parallel construct
  - Finite State Machine Model

Adding Constructs for Parallel Computing



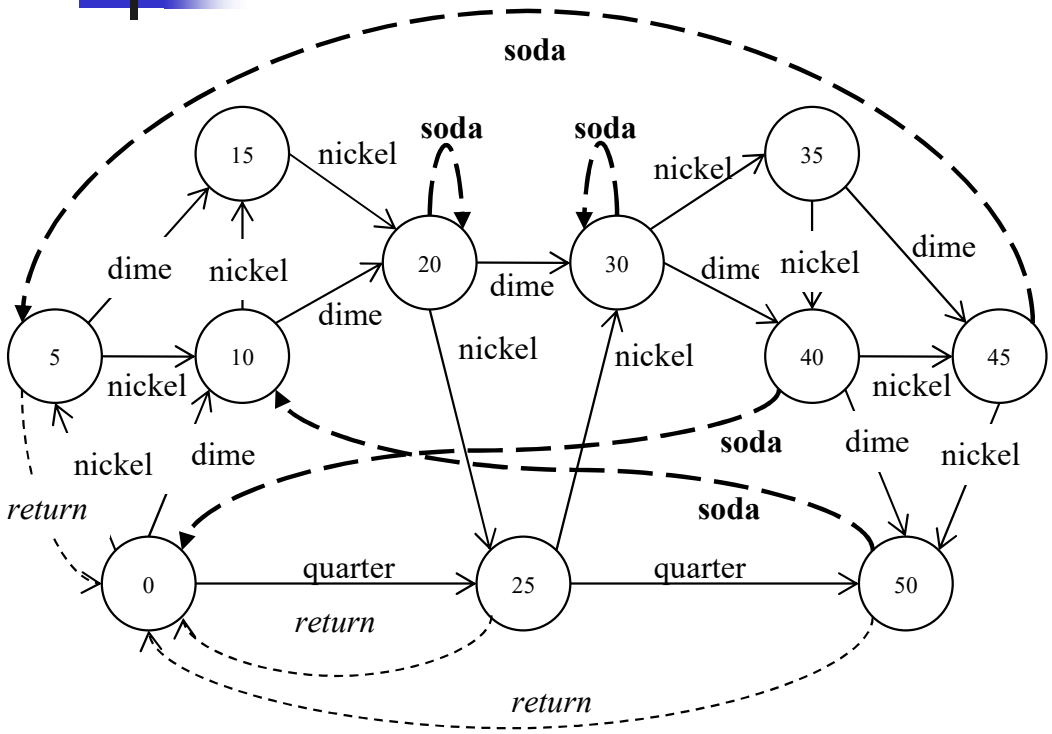

```
Console.WriteLine("Printing from CodeActivity1"); // Thread 1
Console.WriteLine("Hello World"); // Add this line of code
```

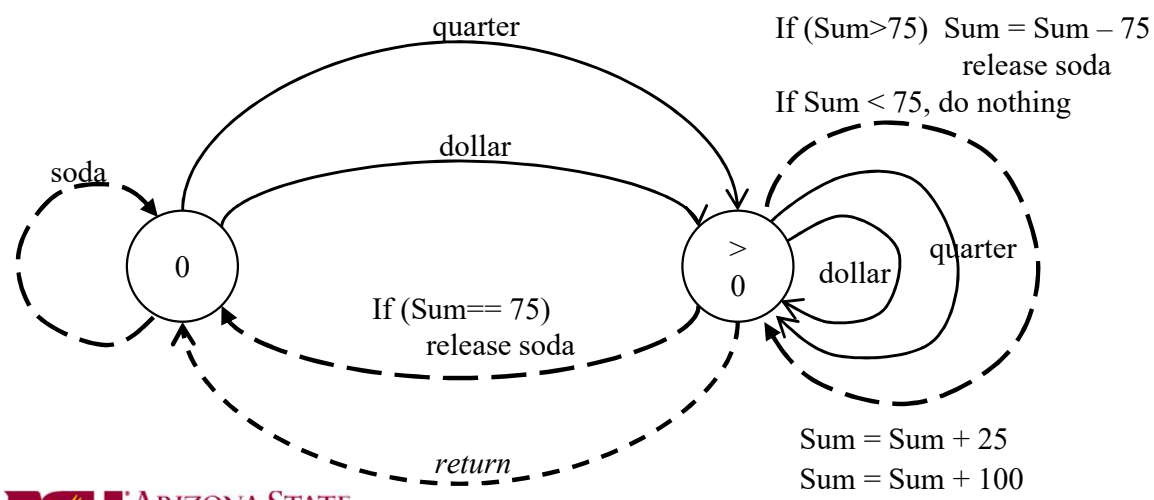
```
Console.WriteLine("Printing from CodeActivity2"); // Thread 2
Double piValue = System.Math.PI;
Console.WriteLine(piValue);
```

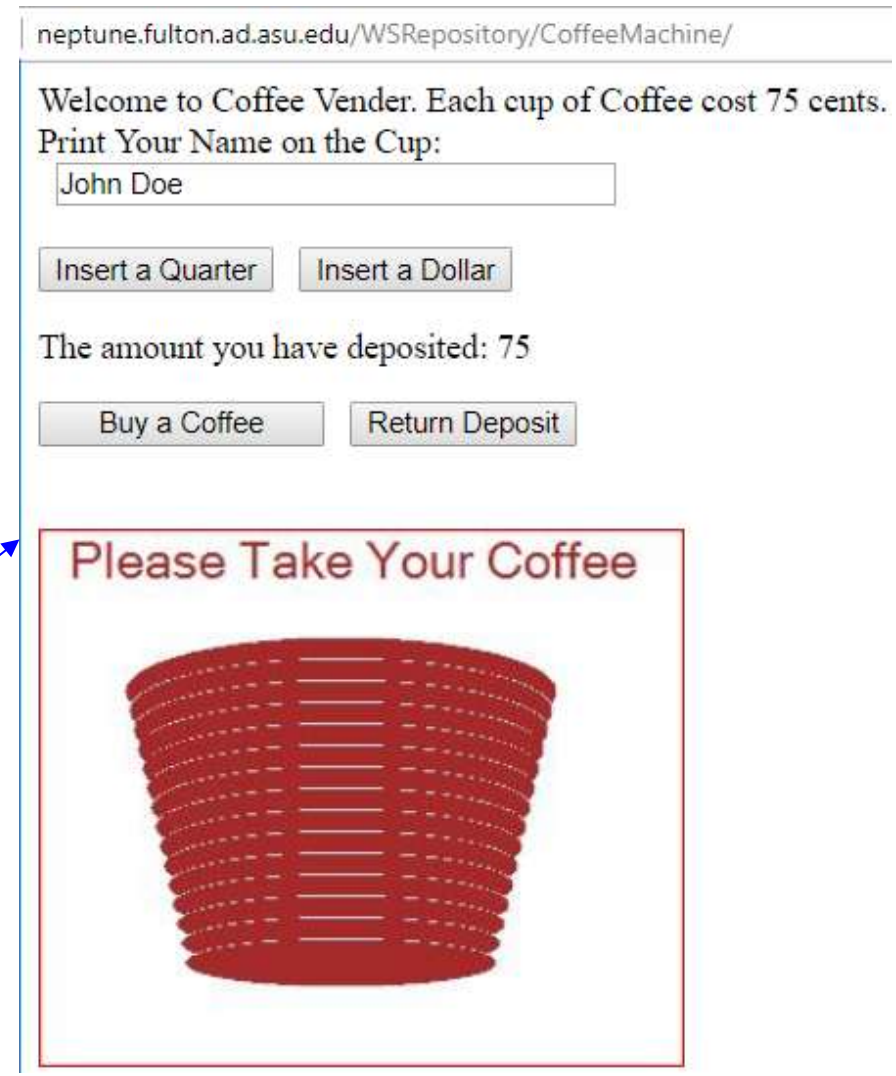


Finite State Machine Model and Event-Driven Programming

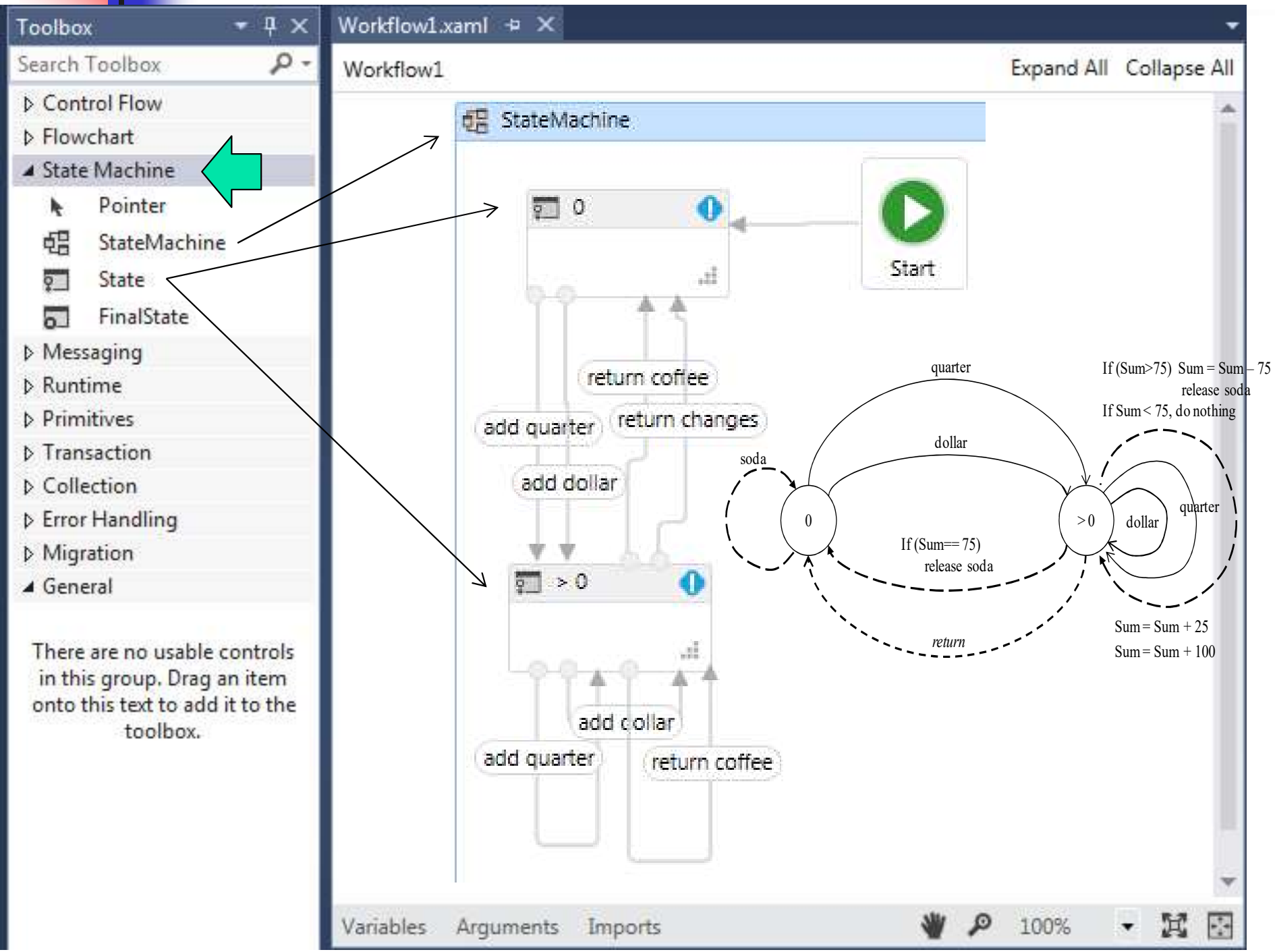
http://venus.sod.asu.edu/WSRepository/CoffeeMachine/







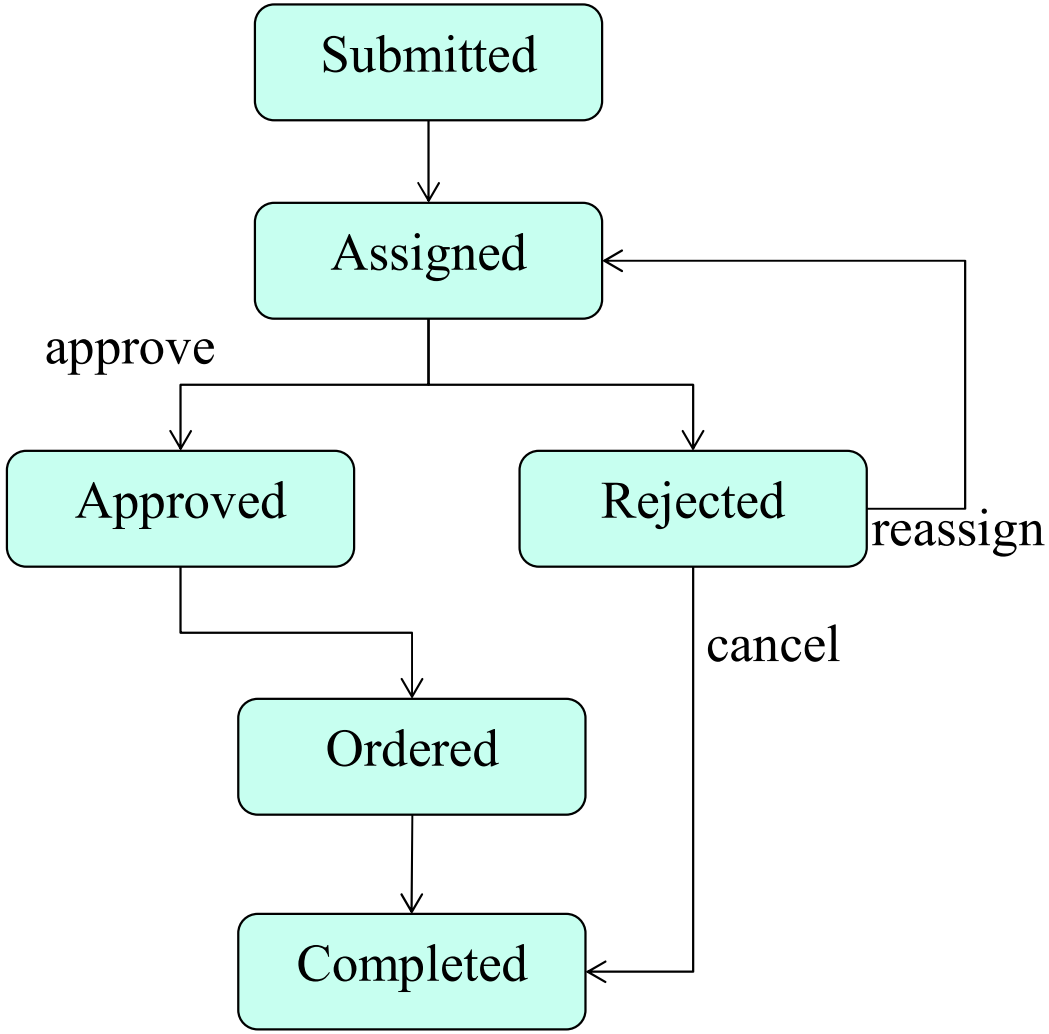
Implementing a State Machine



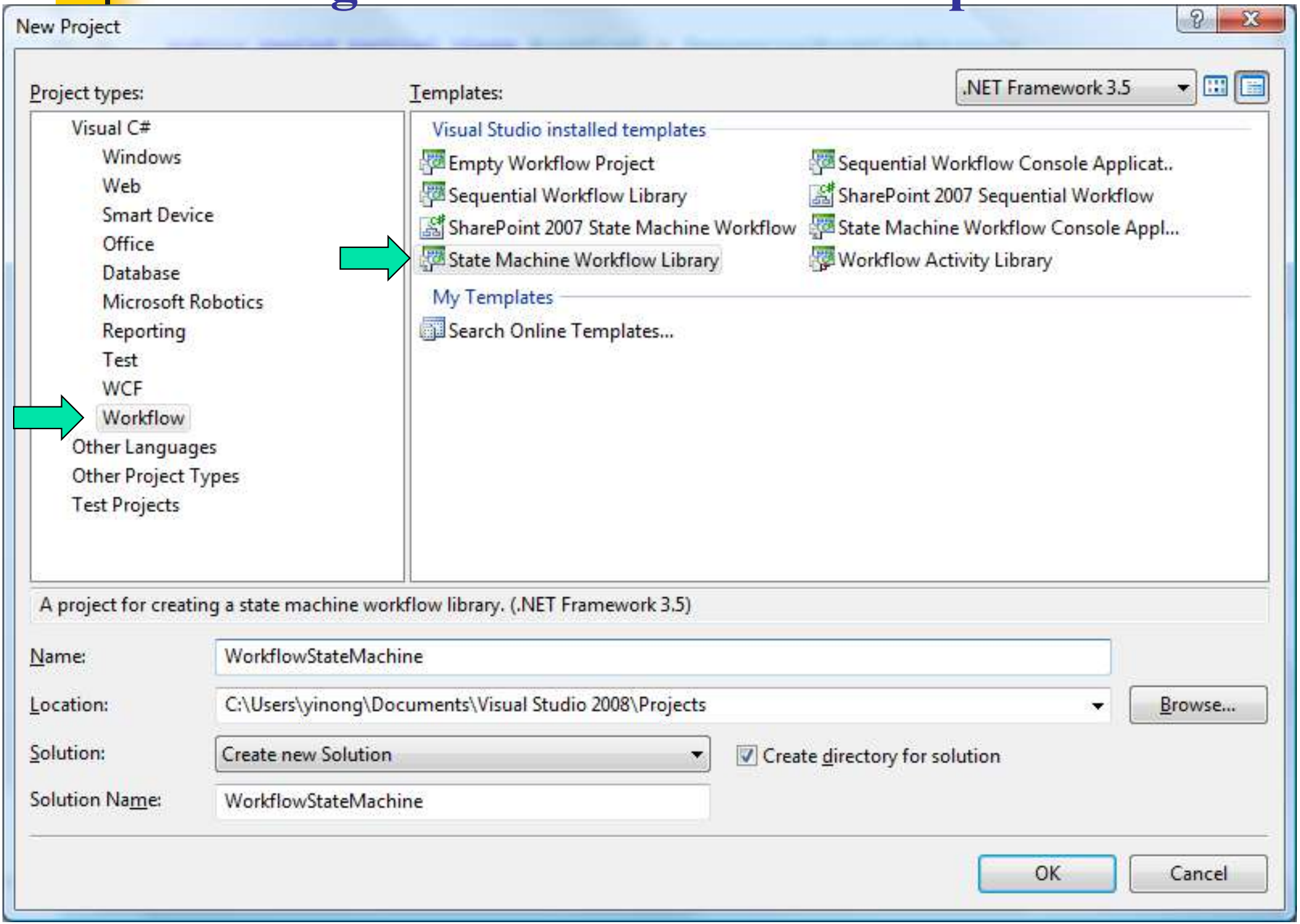
Event-Driven and State Machine for eBusiness Application

http://msdn.microsoft.com/en-us/magazine/cc163281.aspx

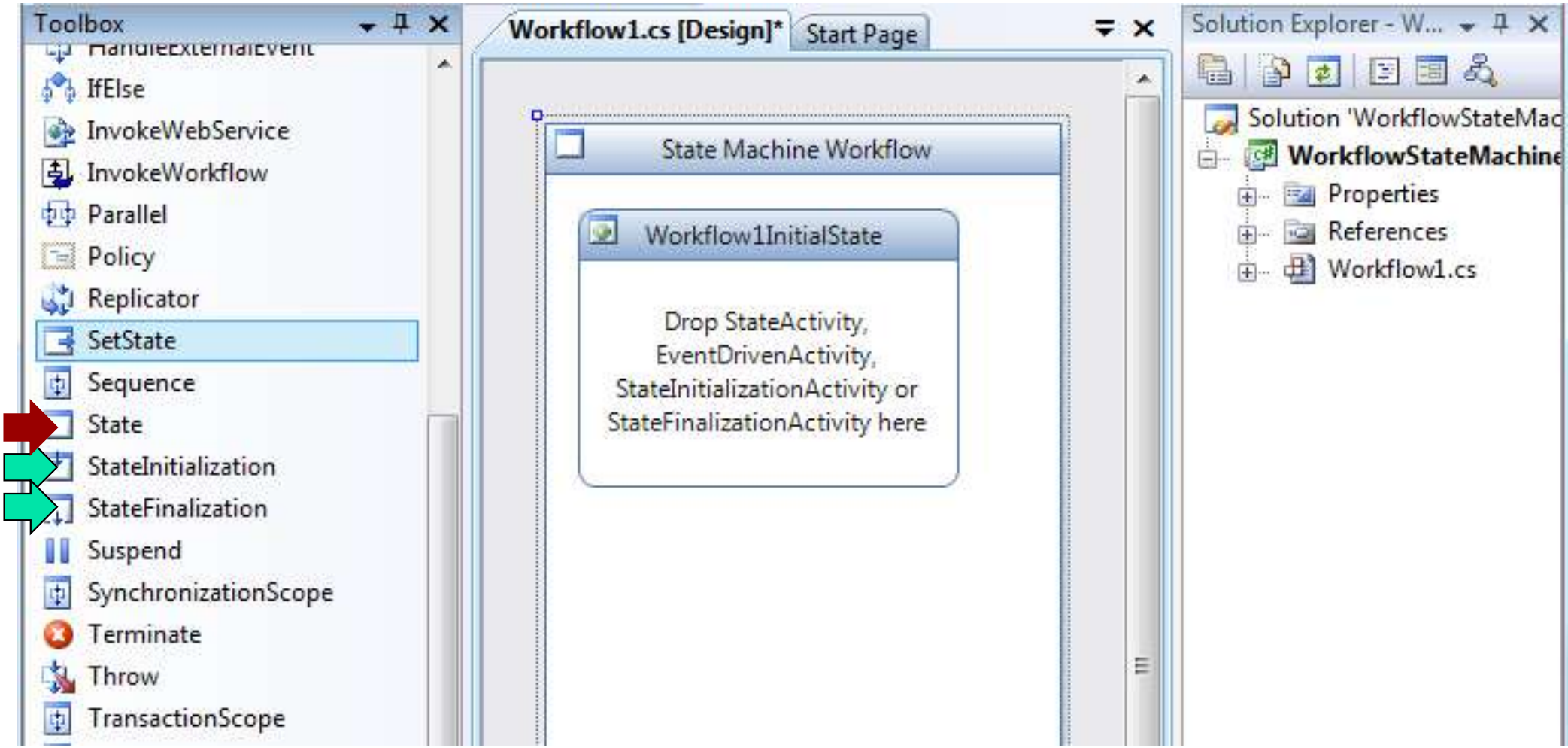
A Simplified Ordering Process



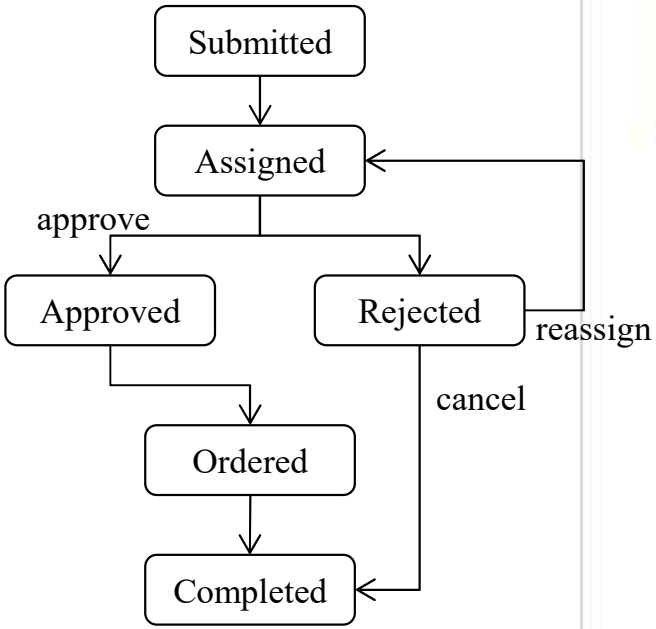
Using the State Machine Template

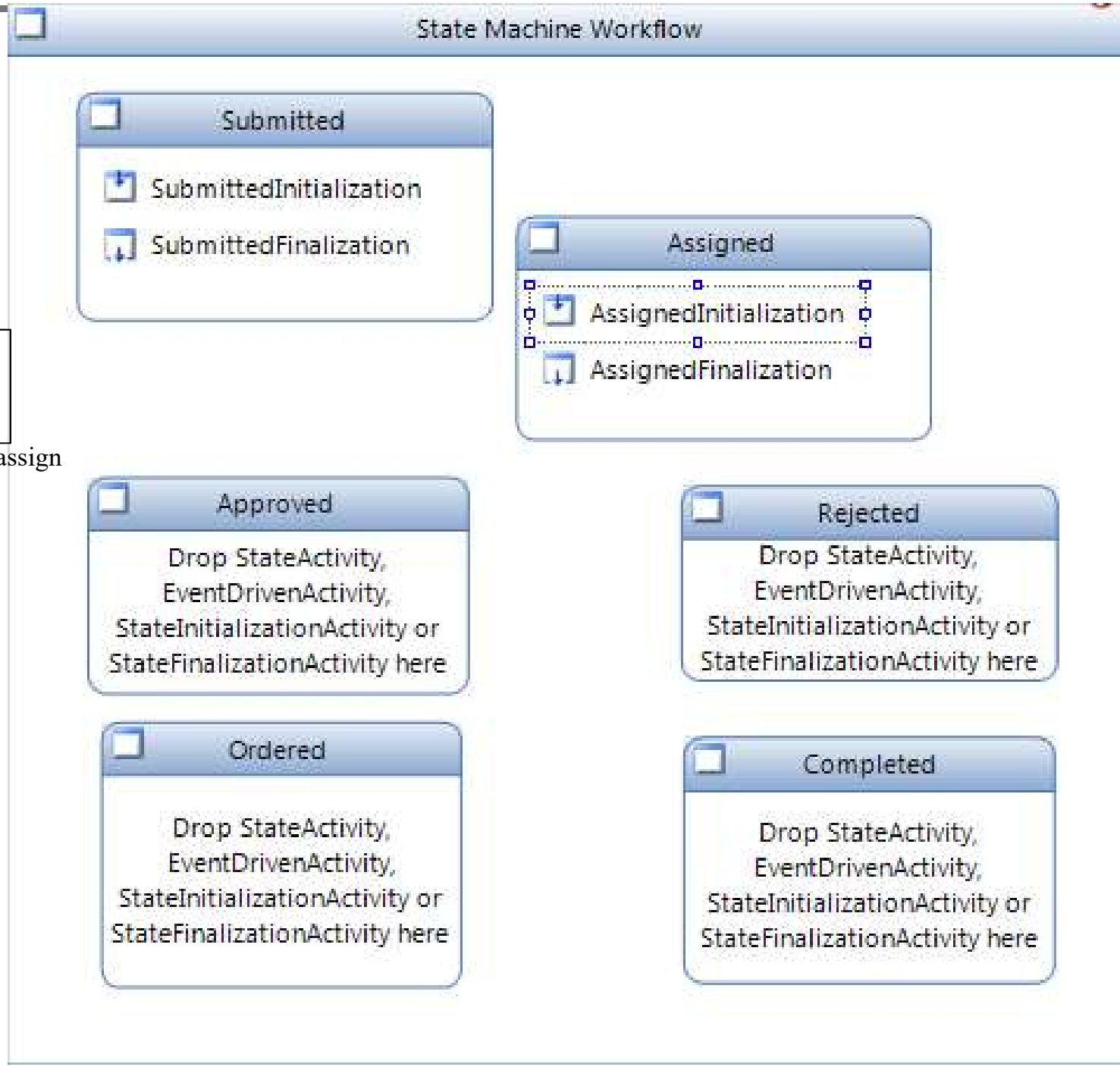


State Machine Template and Services

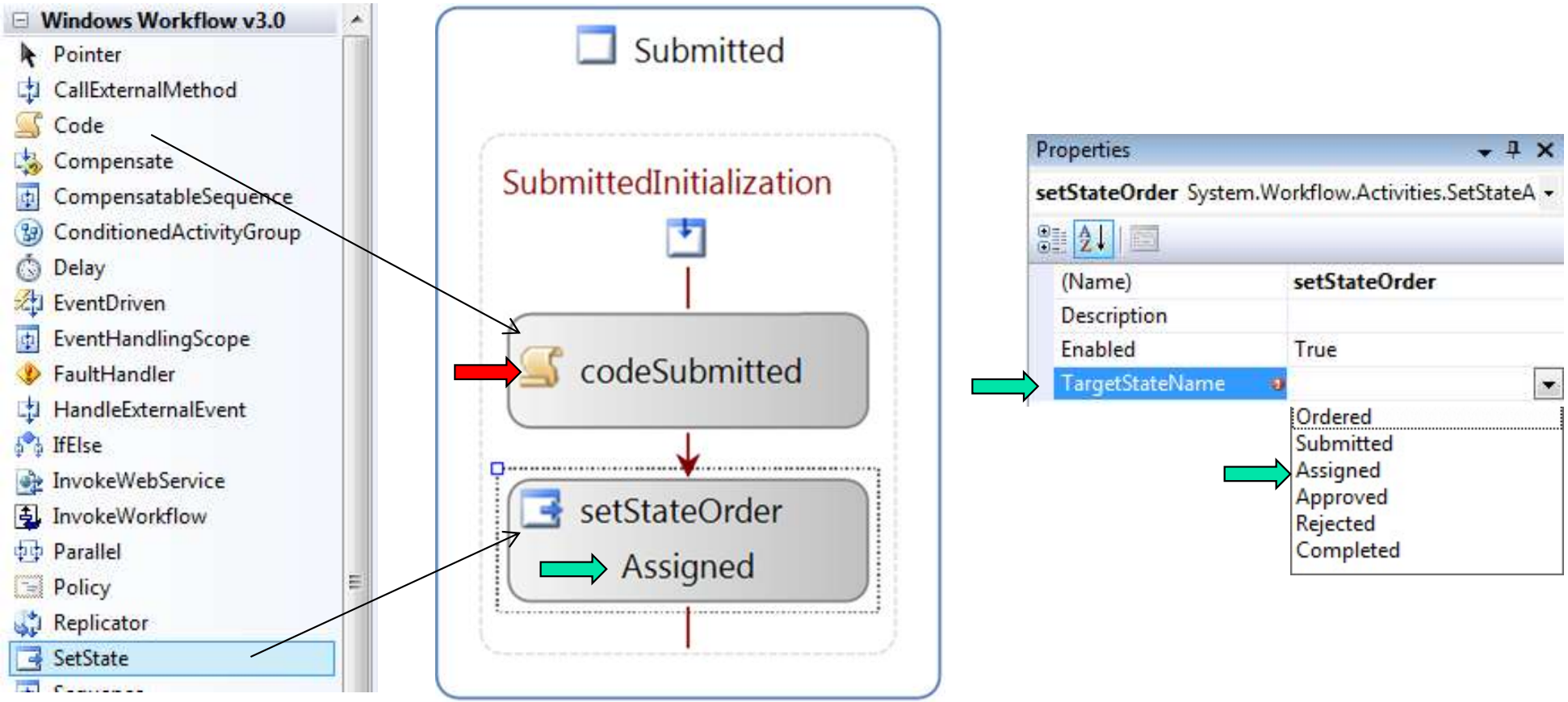


Define States

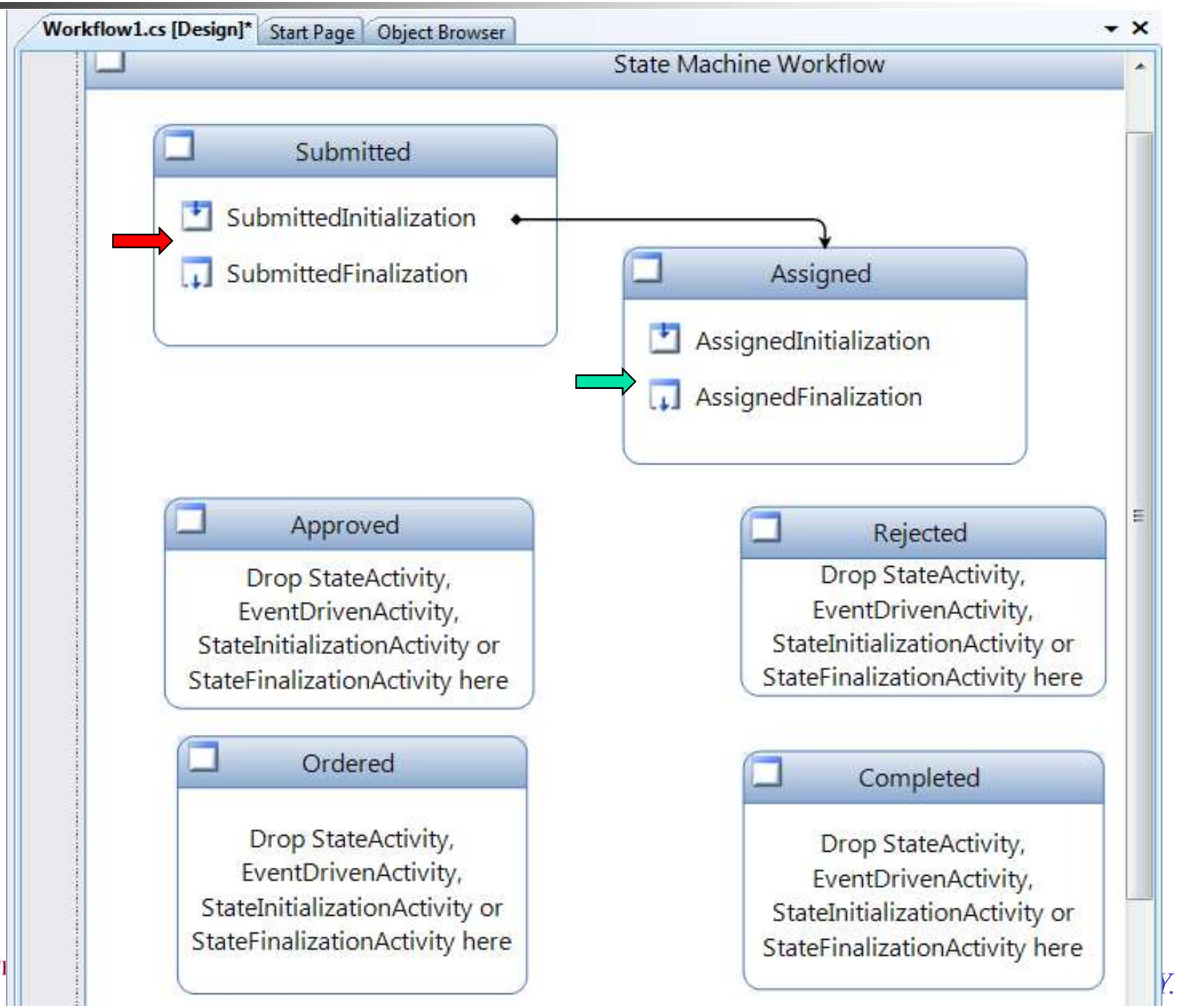




Connecting States Through Target State

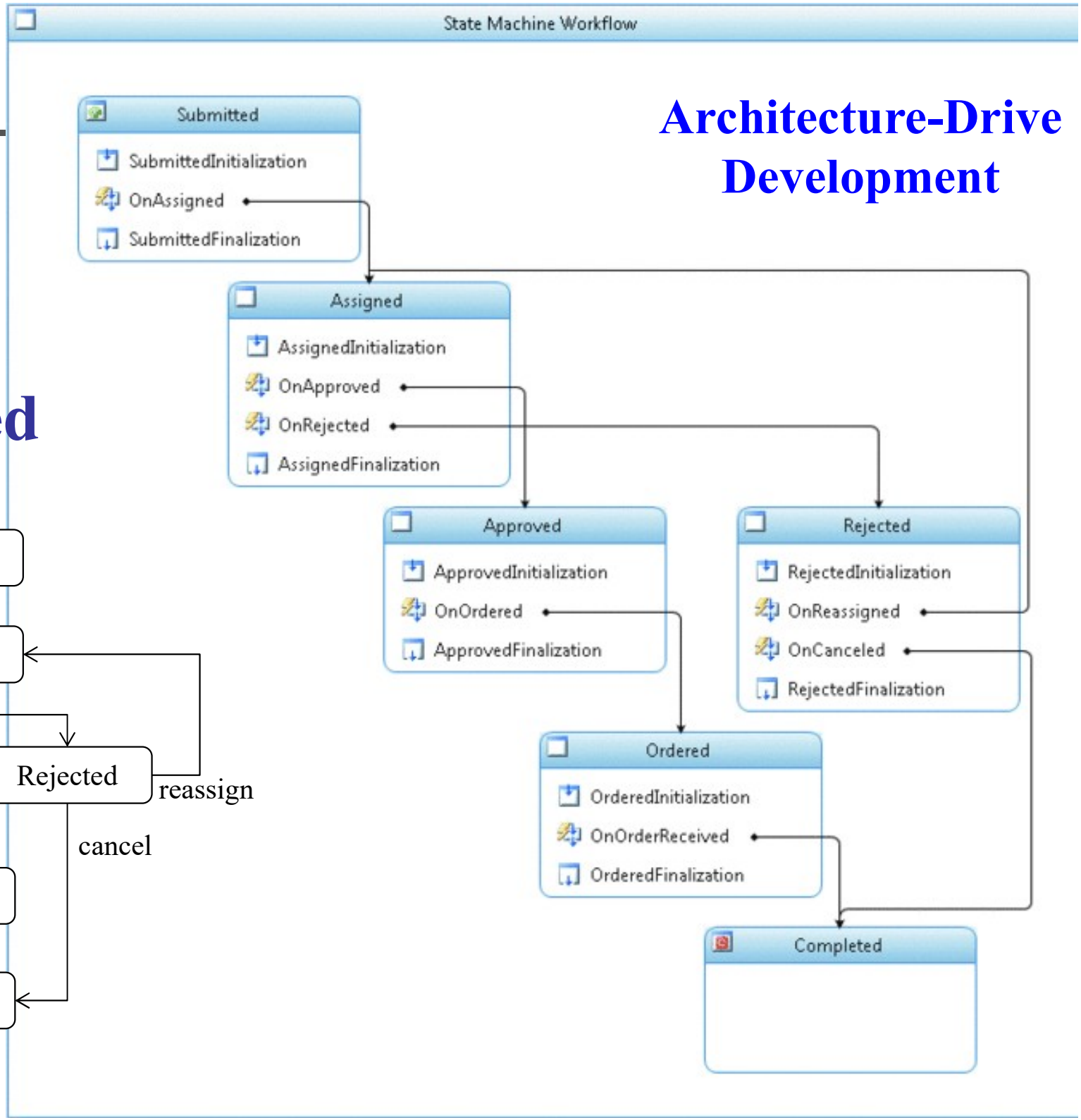


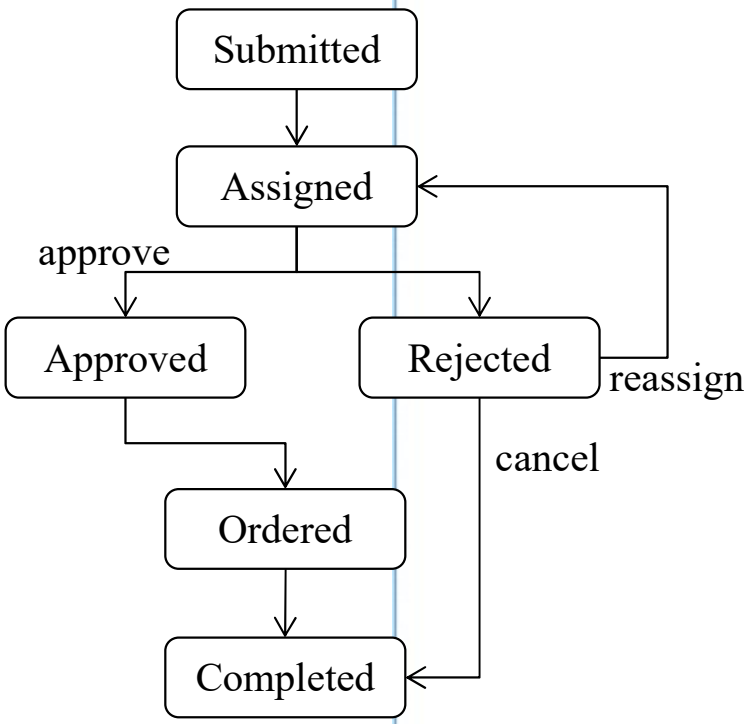
States are Connected



Architecture-Drive Development

All Connected





Summary the Lecture

- Key Ideas of Workflow-based Application Development
- WF Constructs and Activities
- Creating WF workflow Application
- Integrating WF and WCF
- Creating WF Services
- Event-Driven Approach and State Machine in WF
- Workflow Supports
  - Low-Code App Development: Power App on AI/Copilot
  - Architecture-Drive Development
  - Control flow and Dataflow
  - Synchronous and Asynchronous communications

Addition Slides (Not required)

Application Requirement (Scenarios)

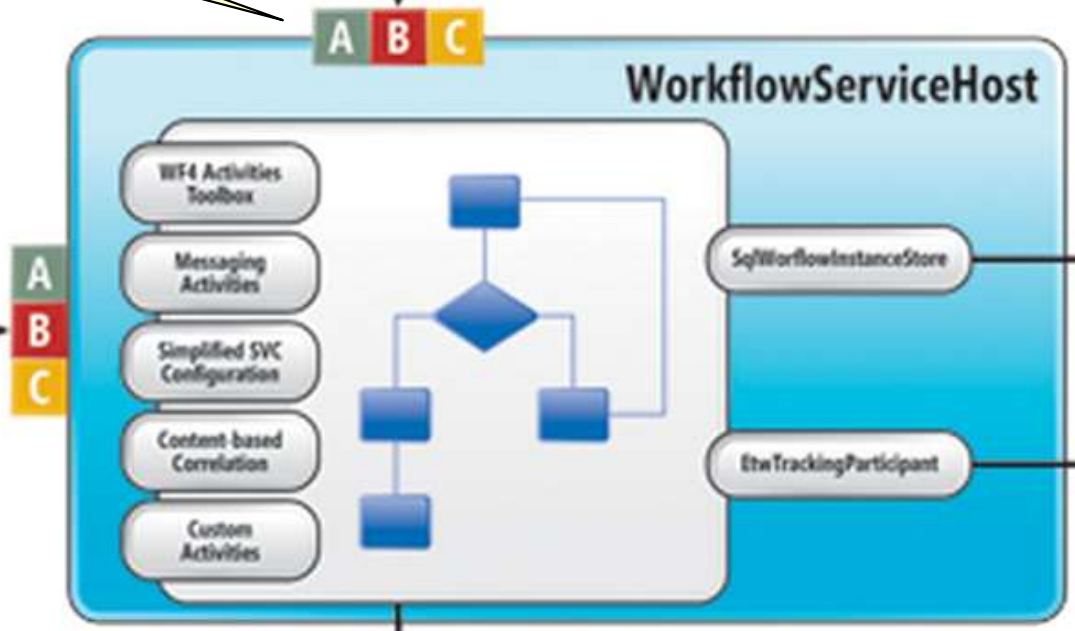
- Contoso Housing is a real estate company that sells houses.
- To provide better customer service and an end-to-end buying experience, Contoso partners with three mortgage companies that assist potential customers with their mortgage needs.
- Each mortgage company offers different interest rates.
- Contoso prioritizes mortgage vendors by their interest rates to ensure that customers get the best deal (using the assumption that a better rate makes the house more likely to sell)

Overview of the Application and Service

Source: http://msdn.microsoft.com/en-us/magazine/ff646977.aspx

What are the ABCs ?

Customers



Case Study: Mortgage Application (Client)

Mortgage App

Home | Ask Mortgage | Application Status | About

Workflow Services

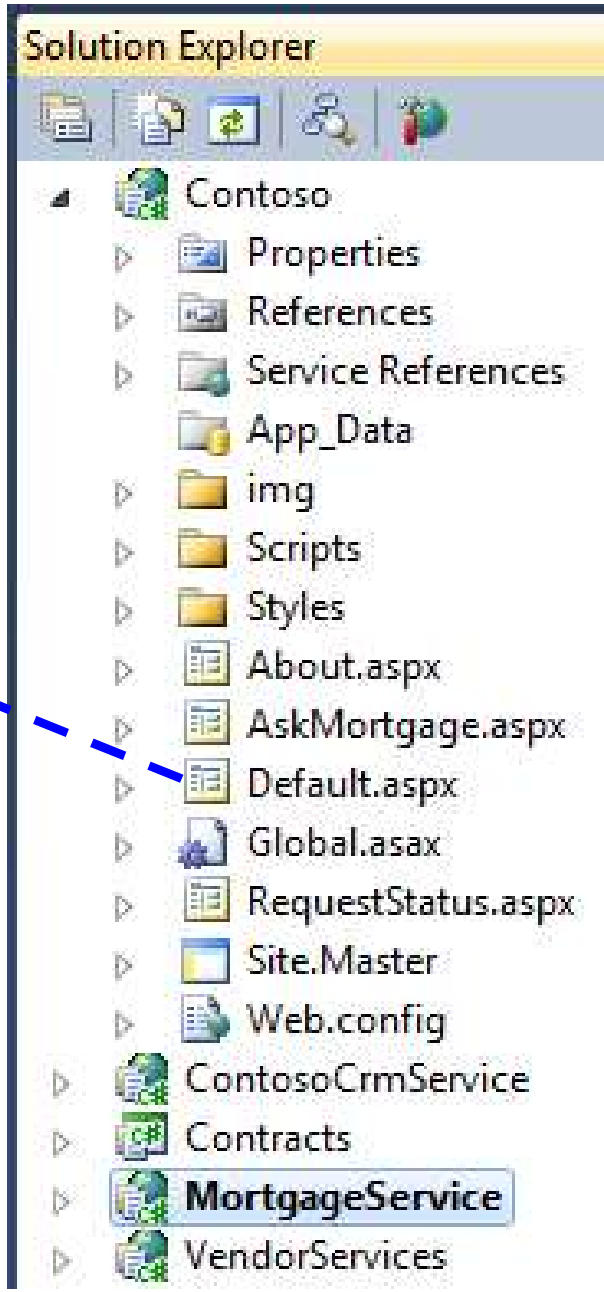
Visual Design of Workflows using WCF and WF 4

Developers are increasingly adopting service-orientated architecture (SOA) as a way of building dis
implementing service-oriented distributed apps can be intimidating. The .NET Framework 4 makes i
Foundation (WCF) services using Windows Workflow Foundation (WF).

WCF workflow services provide a productive environment for authoring long-running, durable opera
ordering of operations is important. Workflow services are implemented using WF activities that car

In this article I will explain how to combine several features of WCF and WF introduced in the .NET F
mortgage approval process for a real estate company without having to write code. This article is n
walk you through the entire process of creating a working solution. Instead, I'm going to focus on th
practical business scenario. A full working solution is included in the code download for this article.

Full article available at MSDN Magazine.



Source: http://msdn.microsoft.com/en-us/magazine/ff646977.aspx

Case Study: Mortgage Application (Client)

Home | Ask Mortgage | Application Status | About

You can combine the image verifier here.

Request Mortgage

General Info

Enter your customer identification: cse445598

House Price: 300000

% Down payment: 20

Years: 30

Salary (yearly): 100000

Do you have credit history in the US? ☑

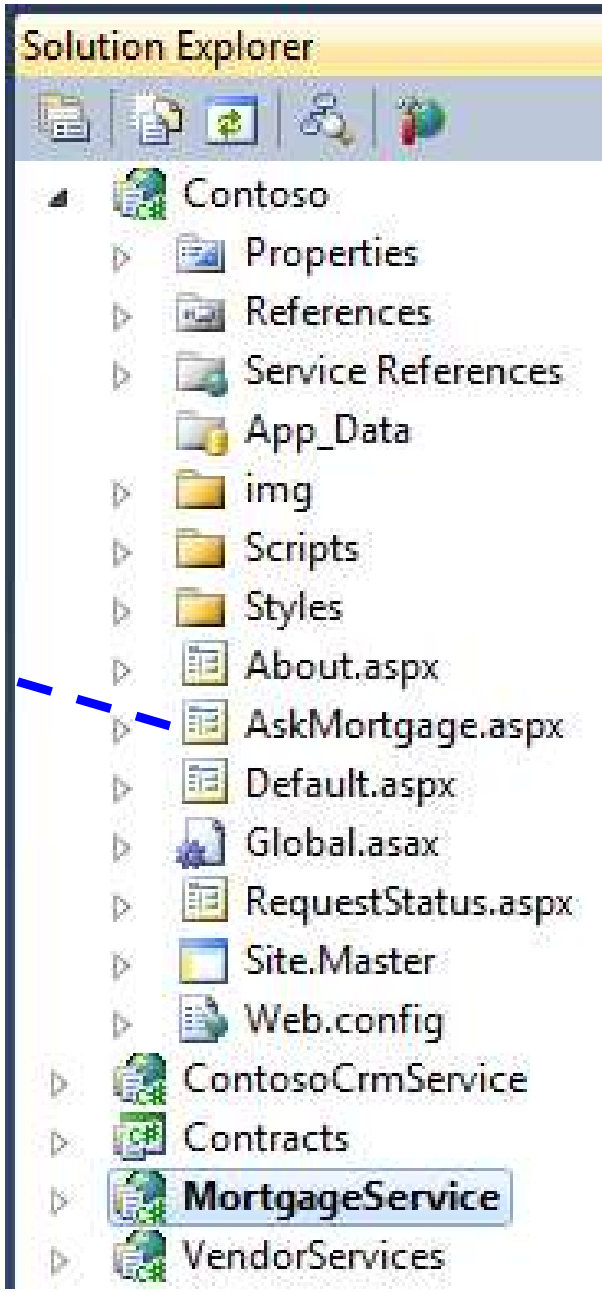
Declarations

☐ Have you been declared bankrupt within the past 7 years?

☐ Have you had proprerty foreclosed upon or given title or deed in lieu thereof in the last 7 years?

☐ Are you a party of a lawsuit?

Start Approval



Case Study: Mortgage Application (Client)

MORTGAGE APP

Home | Ask Mortgage | Application Status | About

CHECK THE STATUS OF YOUR REQUEST

Enter your customer identification cse445598 Update Status

Click 'Update status' to retrieve the status of your application

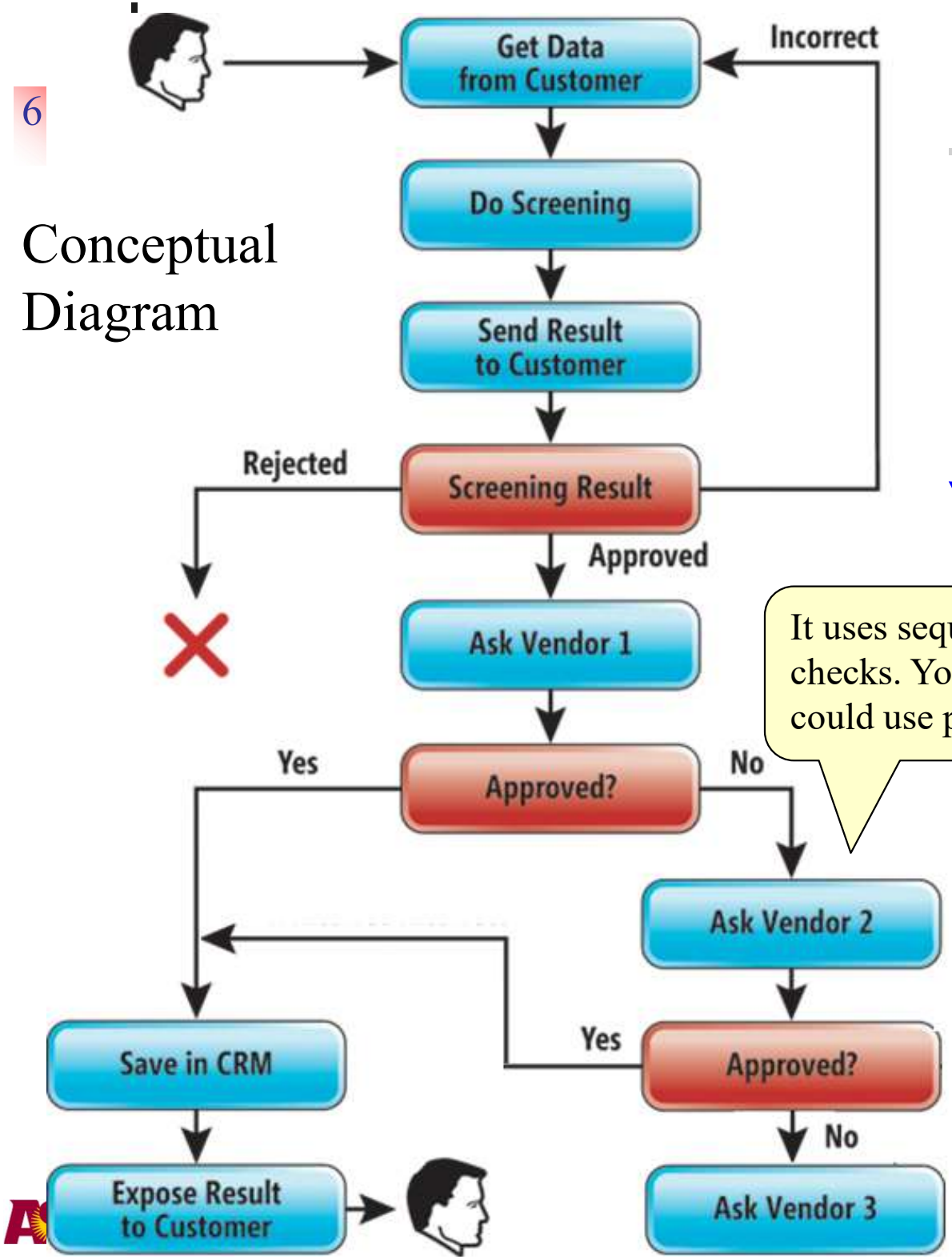
What is a persistence service?

Solution Explorer

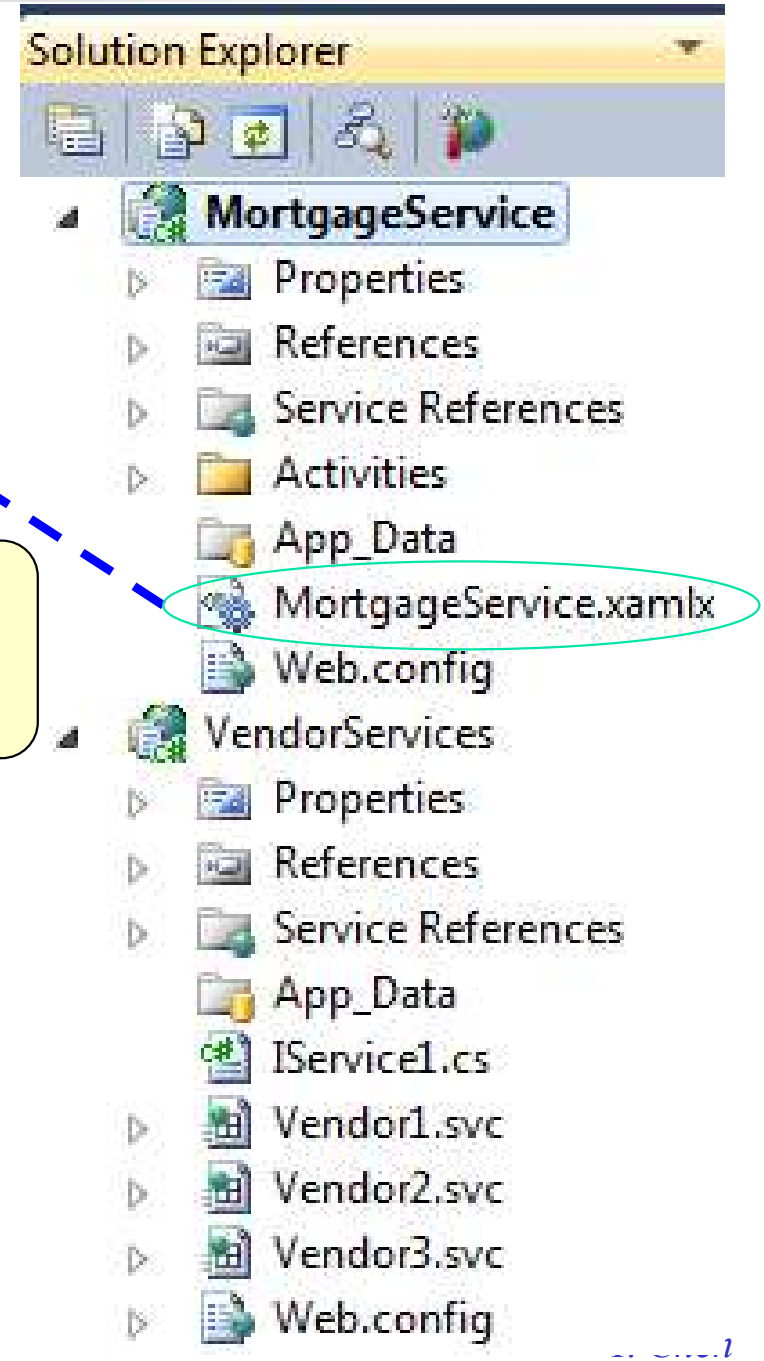
- Contoso
  - Properties
  - References
  - Service References
  - App_Data
  - img
  - Scripts
  - Styles
  - About.aspx
  - AskMortgage.aspx
  - Default.aspx
  - Global.asax
  - RequestStatus.aspx
  - Site.Master
  - Web.config
- ContosoCrmService
- Contracts
- **MortgageService**
- VendorServices

Workflow Service

Conceptual Diagram

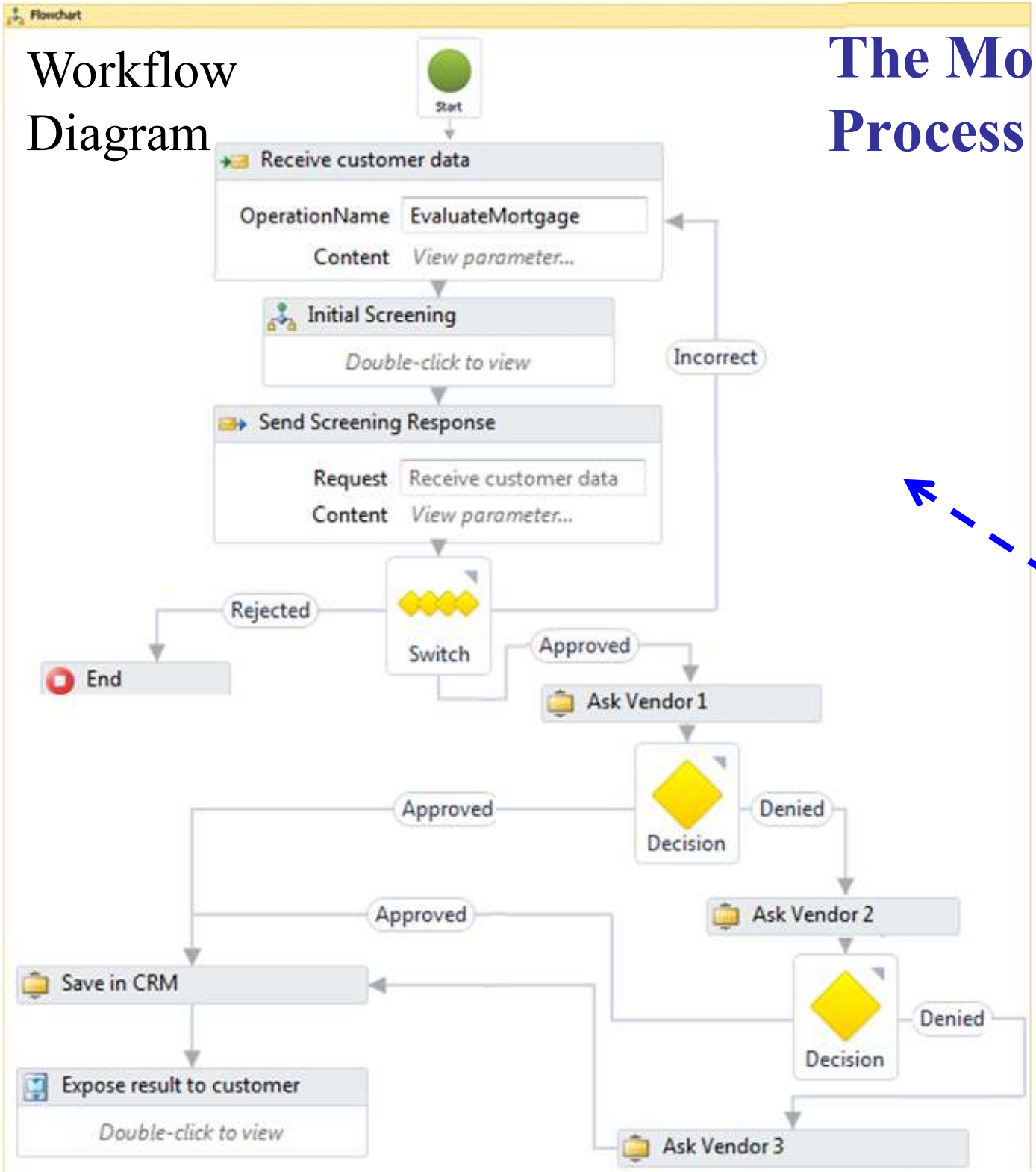


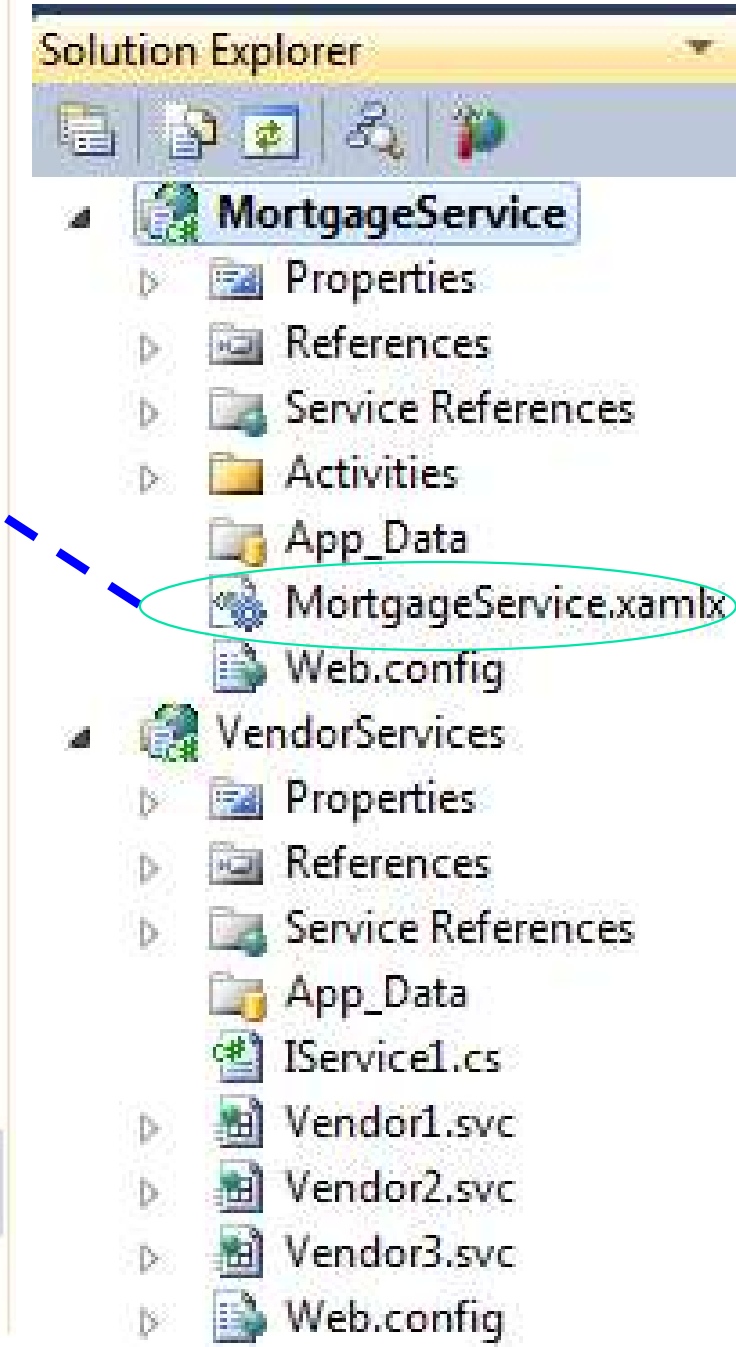
It uses sequential checks. You could use parallel



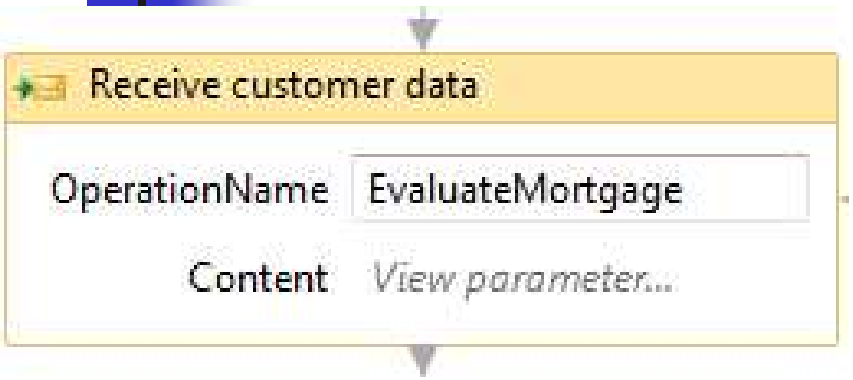
The Mortgage Approval Process as a Flowchart

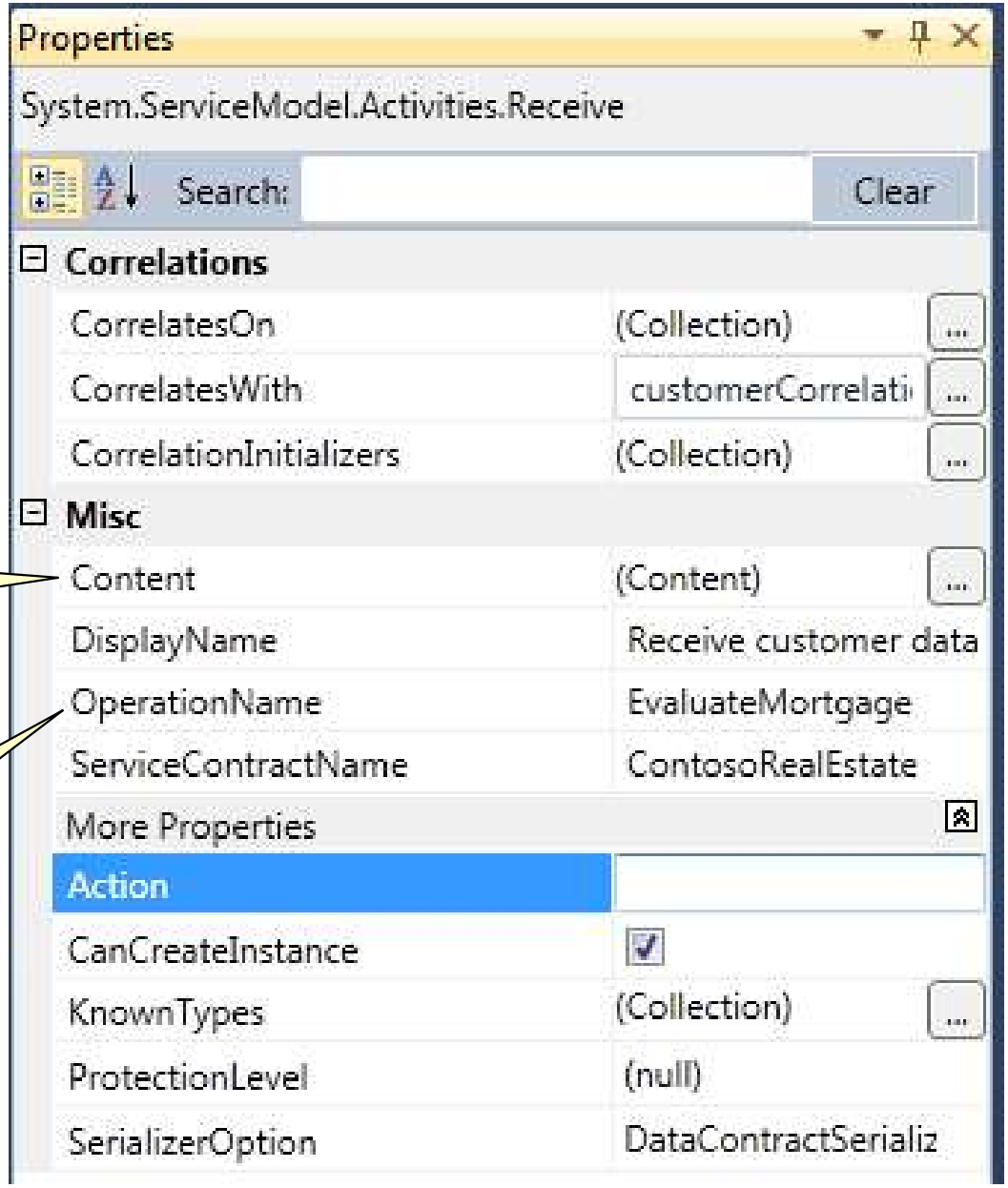
Workflow Diagram





Define the Properties of Receive Activity





Define the WCF Endpoint:
Operation Contract:
Parameters in & out

Define the WCF Endpoint:
Operation Contract:
Operation Name

Configuring Input Parameters

Content Definition

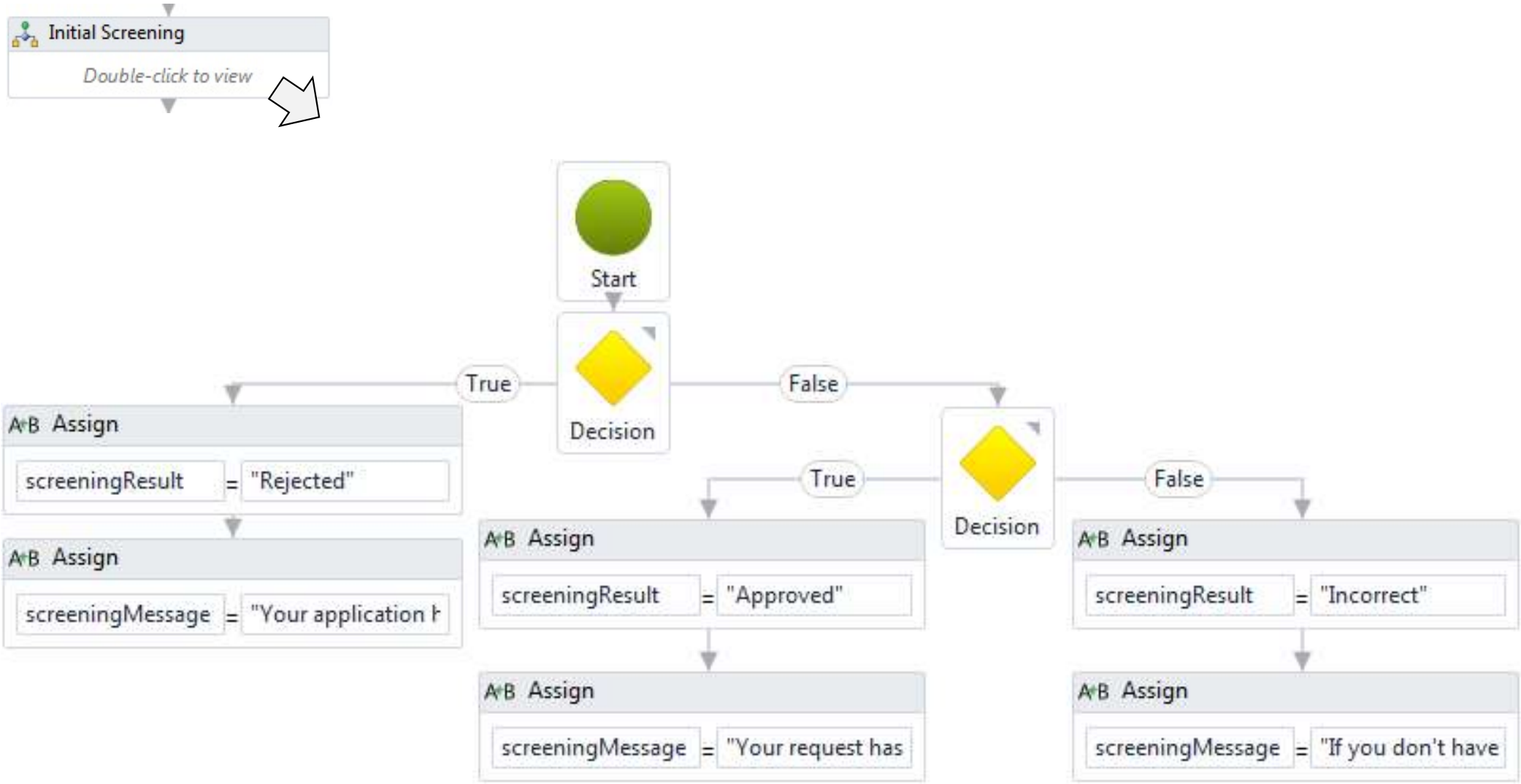
Message

Parameters

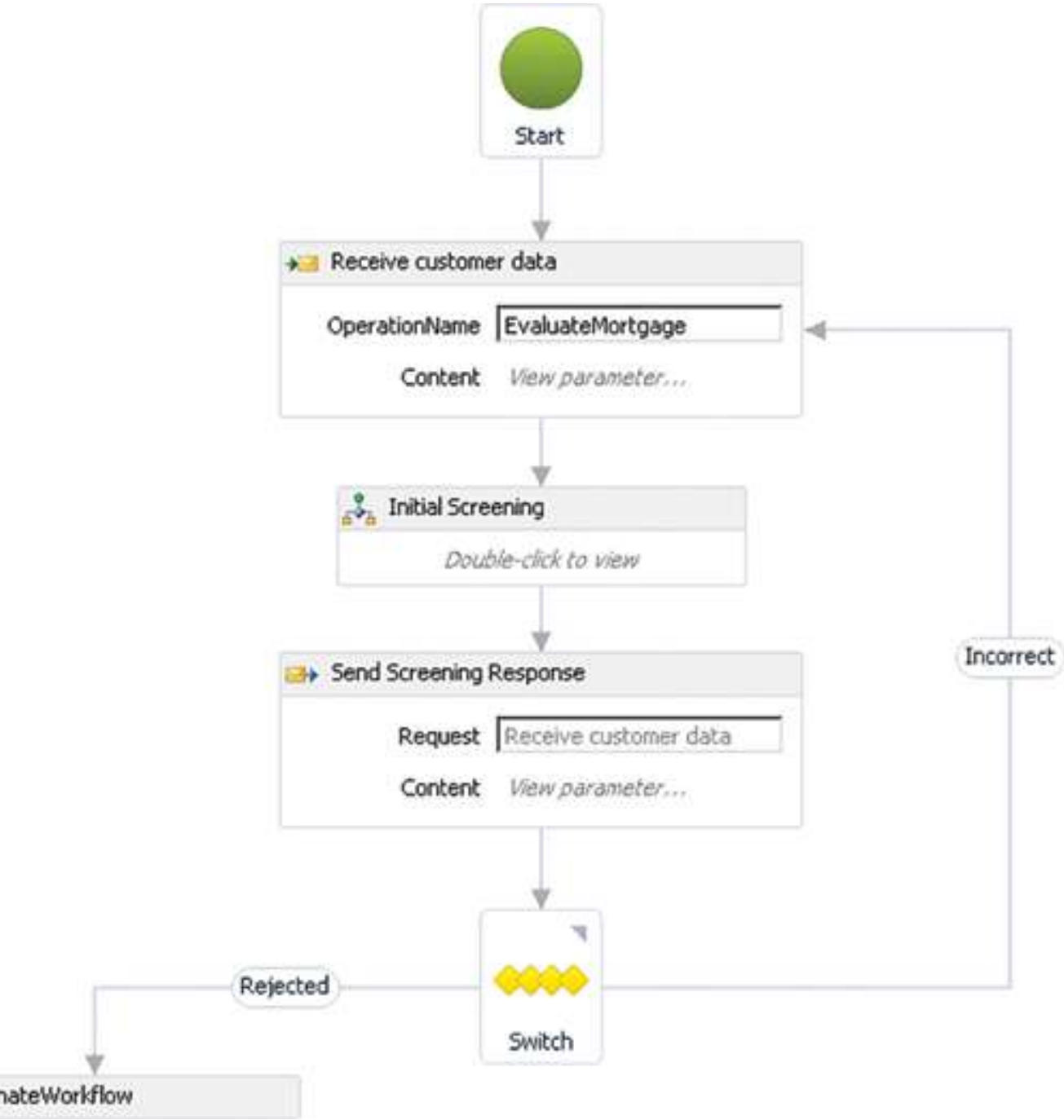
Name	Type	Assign To
customerCode	String	customerId
housePrice	Int32	housePrice
downPayment	Int32	downPercentage
years	Int32	years
salary	Int32	salary
hasCreditHistory	Boolean	hasCreditHistory
hadBankrupcy	Boolean	hadBankrupcy
inLawsuit	Boolean	inLawsuit
hadForeclosure	Boolean	hadForeclosure
Add new parameter		

OK Cancel

Adding the "Initial Screening" Flowchart



Each Outbound Arrow in the Switch Represents a Case in the Switch



Ask Vendor CodeActivity

Ask Vendor

Sequence

Send
OperationName AskRate
Content View parameter...

ReceiveReplyForSend
Request Send
Content Define...

Receive
OperationName SetRate
Content View parameter...

If
Condition
rate > 0
Then
Assign
result = "Mortgage approve
Else
Assign
result = "Mortgage request

Assign
completed = True

While
Condition
Not completed
Body
Sequence

Receive
OperationName GetState
Content View parameter...

SendReplyToReceive
Request Receive
Content View parameter...

Service Deployed at

http://venus.sod.asu.edu/WSRepository/Services/WFService/MortgageService/Service1.xamlx

venus.sod.asu.edu/WSRepository/Services/WFService/MortgageService/Service1.xamlx

Service1 Service

You have created a service.

To test this service, you will need to create a client and use it to call the service. You can do this using the svcutil.exe tool from the command line with th

```
svcutil.exe http://venus.sod.asu.edu/WSRepository/Services/WFService/MortgageService/Service1.xamlx?wsdl
```

You can also access the service description as a single file:

```
http://venus.sod.asu.edu/WSRepository/Services/WFService/MortgageService/Service1.xamlx?singleWsdl
```

This will generate a configuration file and a code file that contains the client class. Add the two files to your client application and use the generated clier

C#

```
class Test
{
    static void Main()
    {
        MortgageClient client = new MortgageClient();

        // Use the 'client' variable to call operations on the service.

        // Always close the client.
        client.Close();
    }
}
```

Note that a workflow service generates a WSDL interface!

Testing the Mortgage Example

- The code can be downloaded at http://msdn.microsoft.com/en-us/magazine/ff646977.aspx
- The code is deployed at:
  The client application (TryIt Page):
  https://venus.sod.asu.edu/WSRepository/WFService/
  Services:
  https://venus.sod.asu.edu/WSRepository/Services/WFService/MortgageService/Service1.xamlx
  https://venus.sod.asu.edu/WSRepository/Services/WFService/VendorService/Vendor1.svc
  https://venus.sod.asu.edu/WSRepository/Services/WFService/VendorService/Vendor2.svc
  https://venus.sod.asu.edu/WSRepository/Services/WFService/VendorService/Vendor3.svc
- Only the basic functions, including the ASP .Net Client App, the workflow-based mortgage service, and the three vendor services are deployed. The CRM and the database services are deployed.